

# **Data Mining Notes**

Academic Year 2026/2027

# Contents

|          |                                                                                          |           |
|----------|------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Data Mining and the KDD Process</b>                                   | <b>1</b>  |
| 1.1      | Foundations, Motivations, and Formal Definition . . . . .                                | 1         |
| 1.2      | The Confluence of Disciplines and the KDD Process . . . . .                              | 3         |
| 1.3      | Taxonomy of Data Mining Tasks . . . . .                                                  | 6         |
| 1.4      | Open Challenges in Modern Data Mining . . . . .                                          | 8         |
| <b>2</b> | <b>Data Understanding I: Data Types, Attributes, and Descriptive Statistics</b>          | <b>10</b> |
| 2.1      | Understanding Data and Dataset Taxonomy . . . . .                                        | 10        |
| 2.2      | Attribute Types and Measurement Scales . . . . .                                         | 12        |
| 2.3      | Data Quality Issues . . . . .                                                            | 14        |
| 2.4      | Univariate Distribution Inspection . . . . .                                             | 14        |
| 2.5      | Measures of Central Tendency and Dispersion . . . . .                                    | 16        |
| 2.6      | Five-Number Summary, Quartiles, and Box Plots . . . . .                                  | 17        |
| <b>3</b> | <b>Data Understanding II – Multivariate Analysis, Correlation, and Outlier Detection</b> | <b>19</b> |
| 3.1      | Multidimensional Visualization Techniques . . . . .                                      | 19        |
| 3.2      | Bivariate Dependence Analysis: Covariance and Correlation .                              | 24        |
| 3.3      | Outlier Taxonomy and Detection Methodologies . . . . .                                   | 28        |
| 3.4      | Introduction to Missing Values . . . . .                                                 | 29        |
| <b>4</b> | <b>Data Preparation &amp; Cleaning – Reduction, Discretization, and Transformation</b>   | <b>31</b> |
| 4.1      | Distinction Between Data Understanding and Data Preparation                              | 31        |
| 4.2      | Aggregation and Sampling . . . . .                                                       | 32        |
| 4.3      | Dimensionality Reduction . . . . .                                                       | 34        |
| 4.4      | Principal Component Analysis (PCA) . . . . .                                             | 36        |
| 4.5      | Handling Missing Values . . . . .                                                        | 39        |
| 4.6      | Discretization and Binarization Techniques . . . . .                                     | 40        |
| 4.7      | Functional Attribute Transformations and Variance Stabilization                          | 42        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 4.8      | Normalization Algorithms . . . . .                             | 44        |
| 4.9      | Univariate Outlier Treatment . . . . .                         | 45        |
| <b>5</b> | <b>Python Lab: Data Understanding and Data Preparation</b>     | <b>48</b> |
| 5.1      | Environment, Ingestion, and Data Integration . . . . .         | 49        |
| 5.2      | Missing Value Diagnosis and Imputation . . . . .               | 51        |
| 5.3      | Multivariate Exploration and Correlation Analysis . . . . .    | 52        |
| 5.4      | Feature Transformation, Discretization, and Encoding . . . . . | 54        |
| 5.5      | Dimensionality Reduction: PCA and <i>t</i> -SNE . . . . .      | 56        |

# Chapter 1

## Introduction to Data Mining and the KDD Process

### 1.1 Foundations, Motivations, and Formal Definition

#### 1.1.1 What is Data Mining?

**Data Mining** (knowledge extraction from data) is the application of efficient computational algorithms and techniques to the analysis of large-scale data collections, aiming to discover previously unknown, non-trivial, valid, and potentially actionable patterns, regularities, correlations, and structures.

Conceptually, Data Mining is neither simple database querying (as in SQL relational queries) nor basic descriptive statistical computation. Its core objective is uncovering **hidden knowledge**: latent dependency structures, implicit models, and predictive relationships that remain undetected through manual inspection or conventional analytical tools.

**Definition 1.1.1** (Data Mining). **Data Mining** is the computational process of automatically or semi-automatically discovering patterns, regularities, correlations, or latent structures within massive datasets (*Big Data*), yielding valid, novel, useful, and interpretable models for human decision-makers.

#### 1.1.2 Application Context: Commercial and Scientific Drivers

The necessity of computational disciplines dedicated to pattern extraction stems from the exponential expansion of data generated, stored, and processed globally:

- **Commercial Perspective:** E-commerce transactions, web browsing logs, retail scanner logs, and financial flows (credit card transactions, wire transfers, POS logs) generate continuous data streams in the order of terabytes or petabytes daily. Modern enterprises and financial institutions cannot gain strategic leverage through passive archival; they require transforming raw logs into behavioral insights for Customer Relationship Management (CRM), personalized recommendations, churn prevention, and real-time fraud mitigation.
- **Scientific Perspective:** Remote sensing and advanced instrumentation produce unprecedented data volumes. Earth observation satellites (e.g., the NASA EOSDIS program) ingest petabytes of environmental measurements yearly; digital sky surveys scan billions of celestial objects; high-throughput genomics generates nucleotide sequences continuously; and computational physical and climate simulations yield terabytes within hours. Scientific hypothesis formulation and empirical analysis inherently require scalable automated synthesis.

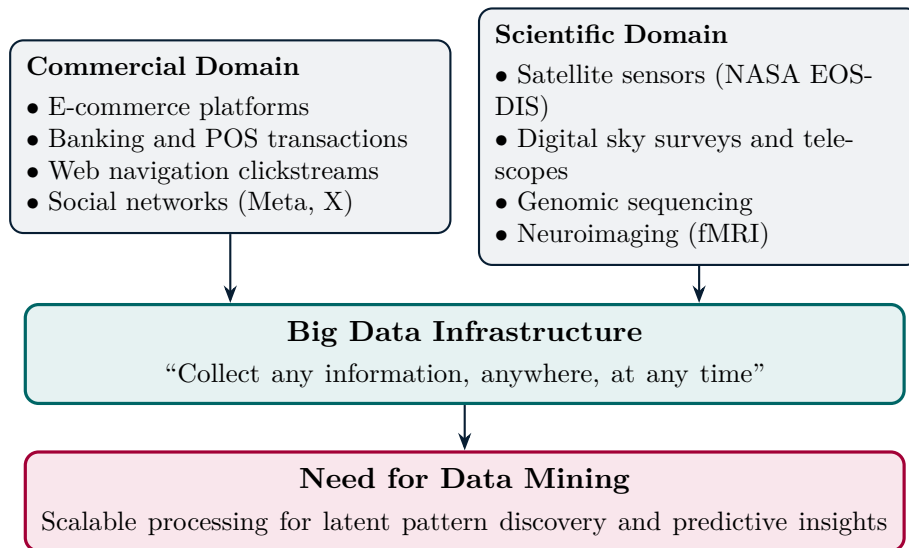


Figure 1.1: Genesis and evolution of Data Mining across commercial and scientific drivers.

### 1.1.3 Big Data as a Proxy for Human Activity and Global Challenges

Digital traces generated by connected devices, mobile nodes, and online services act as an empirical *quantitative proxy* for human behaviors and social

dynamics:

1. **Purchasing Patterns and Lifestyles:** Supermarket receipts, loyalty cards, and online baskets trace dietary habits, price elasticity, and household economics.
2. **Social Networks and Interactions:** Interaction graphs across social platforms reveal interpersonal contact topology and information diffusion dynamics.
3. **Public Sentiment and Collective Mood:** Search queries, microblogging posts, and collaborative encyclopedias measure public interests and sentiment trends in real time.
4. **Mobility and Transportation:** GPS sensors, cellular handovers, and air traffic matrices model mobility at both local levels (urban congestion) and global scales (pandemic pathogen dispersal tracking via platforms such as *BioDiaspora*).

These data-driven techniques enable society to tackle critical global challenges: healthcare efficiency, smart-grid energy optimization, climate change impact forecasting, and agricultural yield planning.

## 1.2 The Confluence of Disciplines and the KDD Process

### 1.2.1 A Multidisciplinary Confluence

Data Mining is inherently cross-disciplinary, emerging at the intersection of several core fields:

- **Database Systems:** Provide the infrastructure for scalable data storage, indexing, secondary-memory access, and distributed retrieval.
- **Statistics:** Furnishes mathematical foundations for uncertainty quantification, sampling, hypothesis testing, and probabilistic modeling.
- **Machine Learning:** Introduces adaptive architectures and optimization algorithms capable of learning empirical rules to generalize on unseen data.

- **Algorithms and Computational Theory:** Ensures analytical procedures remain computationally feasible with respect to time and memory complexities.
- **Visualization:** Enables intuitive perceptual data exploration and the effective communication of analytical findings.

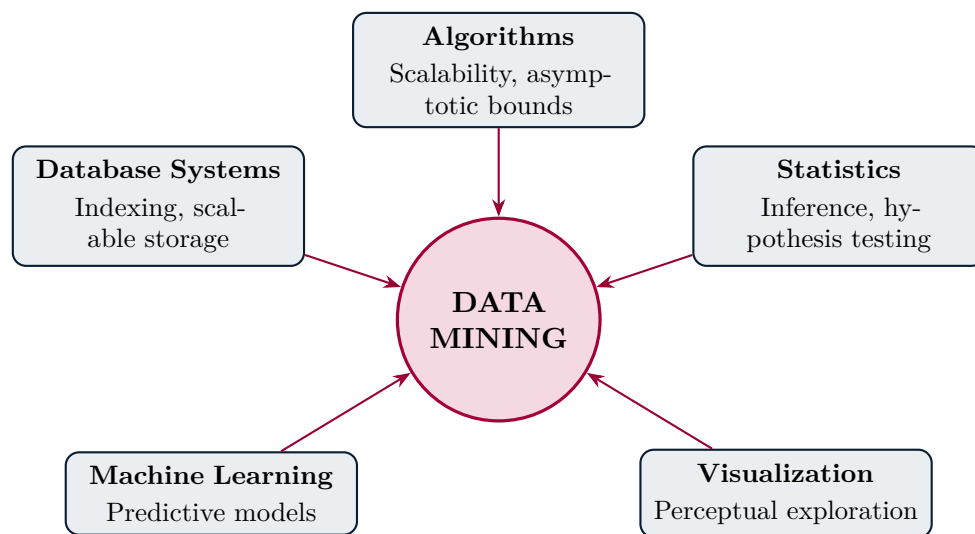


Figure 1.2: The multidisciplinary confluence of Data Mining across computational disciplines.

### 1.2.2 The KDD Process (Knowledge Discovery in Databases)

Data Mining is the algorithmic centerpiece of an iterative, multi-stage engineering methodology known as **KDD** (\*Knowledge Discovery in Databases\*):

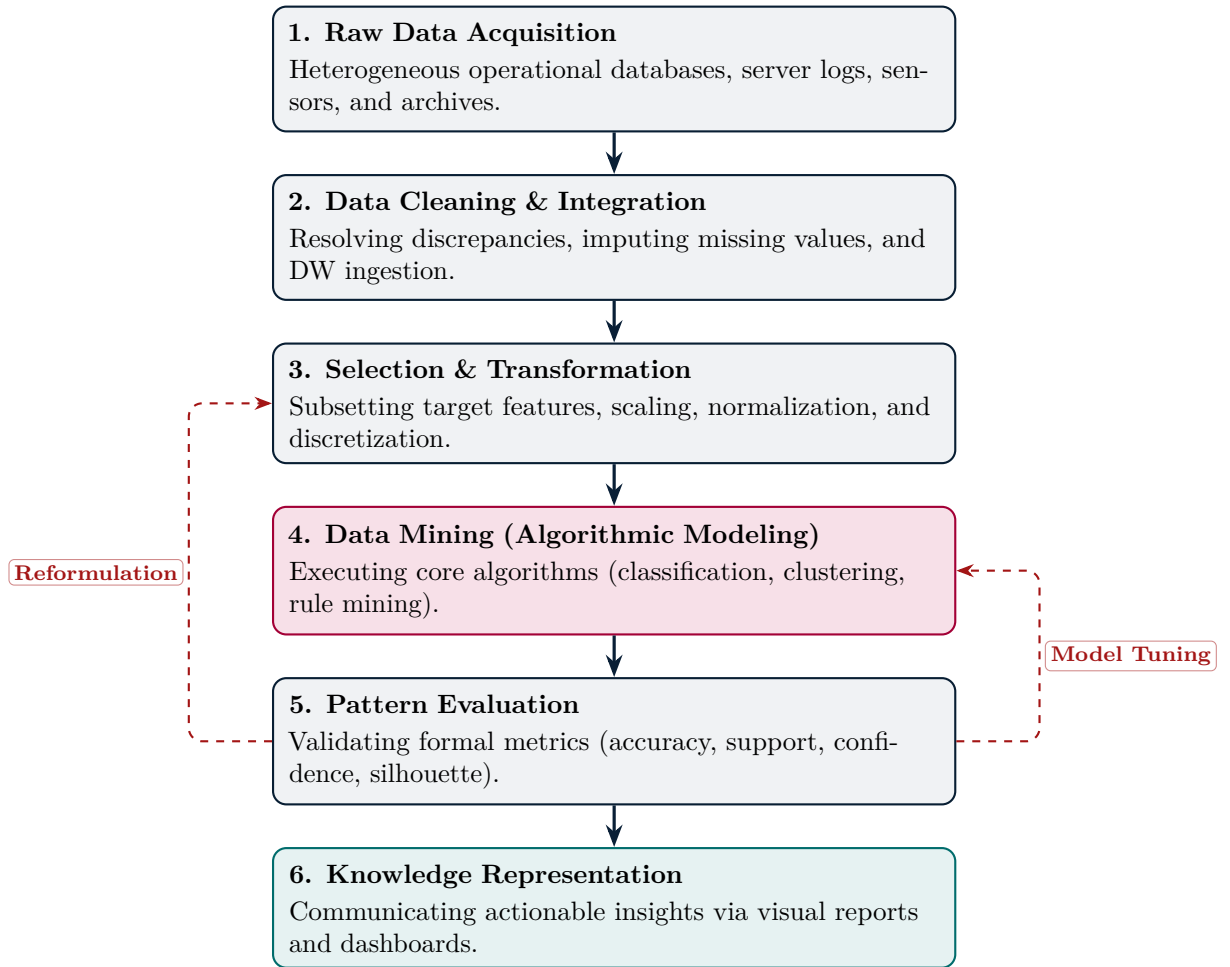


Figure 1.3: Step-by-step workflow of the KDD process with iterative feedback refinement loops.

#### Detailed Phases of the KDD Process:

1. **Business / Goal Understanding:** Formalizing domain objectives, defining KPIs, and setting operational constraints.
2. **Data Cleaning & Integration:** Reconciling fragmented data feeds, correcting noise and errors, and centralizing into a unified schema.
3. **Selection & Transformation:** Filtering target-specific entities and constructing engineered features.
4. **Data Mining:** Deploying supervised or unsupervised algorithms to uncover structural invariants.



5. **Pattern Evaluation:** Validating hypotheses against statistical benchmarks and domain expectations.
6. **Knowledge Representation:** Visualizing and operationalizing findings for stakeholder decision-making.

## 1.3 Taxonomy of Data Mining Tasks

Data Mining tasks are systematically categorized into two macro-classes:

1. **Predictive Methods:** Infer an inductive model mapping input predictors to an unknown continuous or categorical target attribute.
2. **Descriptive Methods:** Explore underlying groupings, correlations, and topological structures without an explicit target variable.

### 1.3.1 Classification (Predictive Modeling)

Classification maps an input vector  $x \in \mathcal{X}$  to a discrete category label  $y \in \mathcal{Y} = \{c_1, c_2, \dots, c_k\}$ . Given a labeled training set:

$$\mathcal{D}_{\text{train}} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} \quad (1.1)$$

the induction algorithm estimates an optimal classification mapping:

$$f : \mathcal{X} \rightarrow \mathcal{Y} \quad (1.2)$$

that minimizes expected classification error on unseen test samples.

**Example: Credit Worthiness Assessment** Consider a toy loan-granting dataset:

| TID | Employed | Level of Education | Years at Address | Credit Worthy (Y) |
|-----|----------|--------------------|------------------|-------------------|
| 1   | Yes      | Graduate           | 5                | <b>Yes</b>        |
| 2   | Yes      | High School        | 2                | <b>No</b>         |
| 3   | No       | Undergrad          | 1                | <b>No</b>         |
| 4   | Yes      | High School        | 10               | <b>Yes</b>        |

Table 1.1: Toy dataset for inductive decision tree credit evaluation.

A decision tree induction algorithm induces the following hierarchical structure:

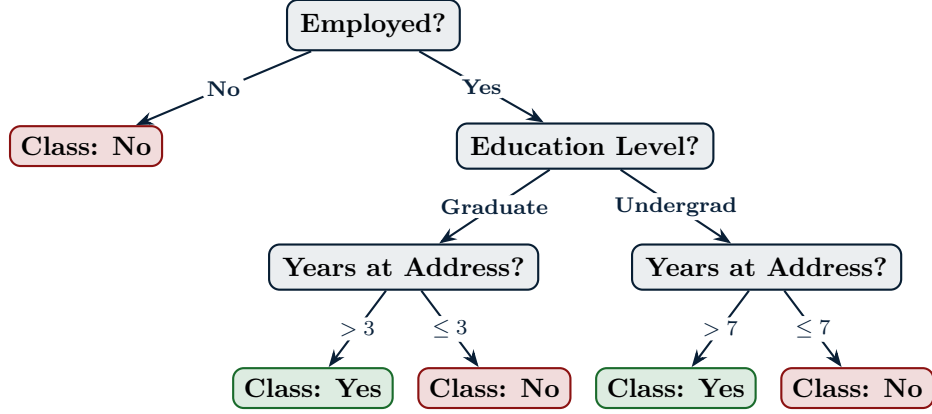


Figure 1.4: Decision tree inferred from the credit worthiness dataset.

### 1.3.2 Clustering (Unsupervised Partitioning)

Clustering partitions an unlabeled dataset  $\mathcal{D} = \{x_1, x_2, \dots, x_N\}$  into  $k$  disjoint subsets  $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$  such that:

1. **Minimized Intra-Cluster Distance:** Items grouped within the same cluster  $C_i$  exhibit high mutual similarity.
2. **Maximized Inter-Cluster Distance:** Items in distinct clusters  $C_i$  and  $C_j$  ( $i \neq j$ ) are well-separated in feature space.



Figure 1.5: Conceptual clustering objective: intra-group cohesion and inter-group separation.

### 1.3.3 Association Rule Discovery

Association analysis identifies co-occurrence affinities within transactional databases. Given an itemset  $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$  and a collection of transactions  $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$  with  $t_j \subseteq \mathcal{I}$ , an **association rule** is defined as:

$$X \implies Y \quad \text{where } X, Y \subset \mathcal{I} \text{ and } X \cap Y = \emptyset \quad (1.3)$$

indicating that transactions containing  $X$  (antecedent) systematically contain  $Y$  (consequent).

| TID | Items Purchased            |
|-----|----------------------------|
| 1   | Bread, Coke, Milk          |
| 2   | Beer, Bread                |
| 3   | Beer, Coke, Diapers, Milk  |
| 4   | Beer, Bread, Diapers, Milk |
| 5   | Coke, Diapers, Milk        |

Table 1.2: Market Basket transaction log.

Prominent extracted rules include:

$$\{\text{Milk}\} \implies \{\text{Coke}\}, \quad \{\text{Diapers, Milk}\} \implies \{\text{Beer}\} \quad (1.4)$$

### 1.3.4 Anomaly Detection (Deviation Detection)

Anomaly detection identifies rare samples or trajectories deviating substantially from standard data generation mechanisms, applied across network intrusion monitoring, payment fraud prevention, predictive equipment failure analysis, and ecological disturbance tracking.

## 1.4 Open Challenges in Modern Data Mining

1. **Scalability:** Sub-quadratic computational requirements to cope with petabyte-scale datasets.
2. **High Dimensionality:** Combating geometric volume inflation and distance concentration.
3. **Complex and Heterogeneous Data:** Harmonizing relational tables, graphs, textual corpora, temporal series, and high-frequency sensor streams.

4. **Distributed Ownership and Privacy:** Complying with regulatory frameworks and privacy constraints across distributed networks.
5. **Dynamic and Non-Stationary Environments:** Accommodating concept drift and online stream learning.

## Chapter 2

# Data Understanding I: Data Types, Attributes, and Descriptive Statistics

### 2.1 Understanding Data and Dataset Taxonomy

**Data Understanding** is the diagnostic investigation in which the analyst examines data quality, understands domain feature semantics, and uncovers initial indications of noise, extreme outliers, or missing values.

#### Core Objectives of Data Understanding

- What types of attributes constitute the dataset?
- What is the overall data quality, and what syntactic/semantic errors exist?
- Are there univariate or multivariate outliers present?
- What pairwise correlations and dependencies connect features?
- How are missing values distributed, and what mechanism generated them?
- Is feature transformation or feature engineering required prior to mining?

### 2.1.1 Fundamental Structure: Objects and Attributes

A dataset is a formal collection of **data objects** (also termed **records**, **points**, **samples**, **entities**, **instances**, or **cases**) described by a set of **attributes** (known as **variables**, **features**, **fields**, or **dimensions**).

- An **object** represents a single real-world entity (a customer, a bank transaction, a sensor reading).
- An **attribute** denotes a measurable property or characteristic of that entity (height, price, temperature, category).

### 2.1.2 Classification of Data Structures

Datasets differ fundamentally by their underlying structural topology:

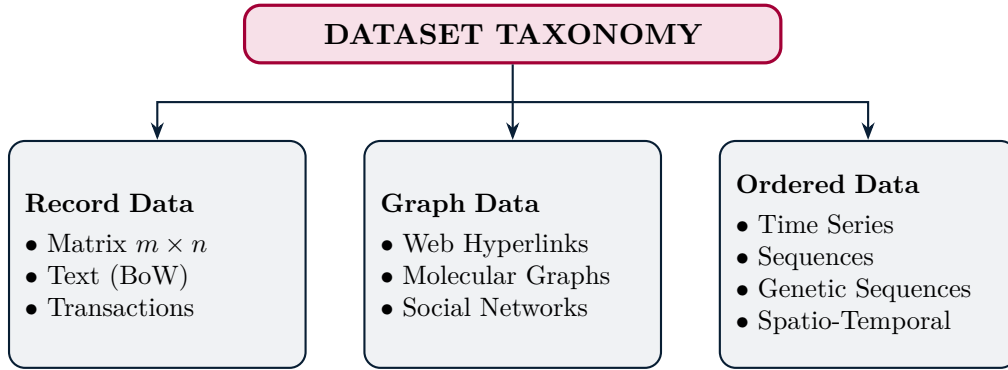


Figure 2.1: Structural taxonomy of major dataset formats.

#### 1. Record Data

- **Data Matrix:** When all  $m$  objects share the same fixed set of  $n$  continuous numerical attributes, the dataset is modeled as an  $m \times n$  matrix:

$$D = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n} \quad (2.1)$$

where each row  $i$  is a point in continuous Euclidean space  $\mathbb{R}^n$ , and each column is a coordinate axis.

- **Document Data (Document-Term Matrix):** Unstructured texts modeled via *Bag-of-Words*. Columns represent vocabulary terms and values record word frequencies:

| Document | team | coach | play | ball | score | game | win | lost | timeout | season |
|----------|------|-------|------|------|-------|------|-----|------|---------|--------|
| Doc 1    | 3    | 0     | 5    | 0    | 2     | 6    | 0   | 2    | 0       | 2      |
| Doc 2    | 0    | 7     | 0    | 2    | 1     | 0    | 0   | 3    | 0       | 0      |
| Doc 3    | 0    | 1     | 0    | 0    | 1     | 2    | 2   | 0    | 3       | 0      |

Table 2.1: Document-Term Matrix representation for text collections.

- **Transaction Data:** Each record is identified by a unique TID and contains a variable-length set of discrete items (e.g., supermarket market basket).

## 2. Graph Data

Explicitly models pairwise entities: World Wide Web hyperlinks, molecular structures (e.g., benzene  $C_6H_6$  covalent bonds), and citation networks.

## 3. Ordered Data

Data where temporal or spatial precedence governs informational semantics:

- **Time Series:** Continuous or discrete time-indexed readings with high autocorrelation.
- **Sequential Data:** Chronologically ordered event sequences (e.g.,  $(A, B) \rightarrow (D) \rightarrow (C, E)$ ).
- **Genomic Sequences:** Nucleotide polymer chains ('A', 'C', 'G', 'T').
- **Spatio-Temporal Data:** Space-time indexed matrices (e.g., global sea-surface temperature grids).

## 2.2 Attribute Types and Measurement Scales

Mathematical admissibility of operations on attribute domains:

- **Distinctness (= or  $\neq$ ):** Testing equality between values.
- **Order (< or >):** Natural total or partial ordering.

- **Difference (+ or −):** Constant-metric measurable intervals.
- **Ratio (\* or /):** Natural absolute zero (total absence of measured property).

| Scale           | Distinct. | Order | Diff. | Ratio | Permissible Statistical Operators        |
|-----------------|-----------|-------|-------|-------|------------------------------------------|
| <b>Nominal</b>  | ✓         | –     | –     | –     | Mode, entropy, $\chi^2$ test             |
| <b>Ordinal</b>  | ✓         | ✓     | –     | –     | Median, quantiles, Spearman rank $\rho$  |
| <b>Interval</b> | ✓         | ✓     | ✓     | –     | Mean, standard deviation, Pearson $r$    |
| <b>Ratio</b>    | ✓         | ✓     | ✓     | ✓     | Geometric mean, coefficient of variation |

Table 2.2: Hierarchy and mathematical properties of measurement scales.

### 2.2.1 Categorical (Qualitative) Attributes

1. **Nominal:** Allows only equality comparison ( $=, \neq$ ). Examples: eye color, zip code, tax ID.
2. **Binary:** Nominal attributes with exactly two values  $\{0, 1\}$ .
  - *Symmetric Binary:* Both outcomes carry equal weight (e.g., gender Male/Female).
  - *Asymmetric Binary:* One outcome is rare or of focal interest (1 = positive medical test, 0 = negative).
3. **Ordinal:** Natural ordering ( $<, >$ ), but step distance is non-constant. Examples: Mohs hardness scale, ratings {poor, fair, good}, letter grades  $\{A, B, C, D, F\}$ .

### 2.2.2 Numeric (Quantitative) Attributes

1. **Interval-Scaled:** Differences are meaningful, but the scale **\*\*lacks an absolute zero\*\***. Examples: Celsius or Fahrenheit temperature (20°C is not “twice as hot” as 10°C, since in Kelvin  $293.15\text{ K} \neq 2 \cdot 283.15\text{ K}$ ); calendar dates.



2. **Ratio-Scaled:** Possesses a **\*\*natural absolute zero\*\***. Examples: length, weight, age, Kelvin temperature (0 K absolute zero), monetary counts. Stating that a 3-hour match is 50% longer than a 2-hour match ( $3/2 = 1.5$ ) is mathematically valid.

### 2.2.3 Discrete vs Continuous Variables

- **Discrete Attribute:** Finite or countably infinite set of values, represented by integers.
- **Continuous Attribute:** Real-valued domain ( $\mathbb{R}$ ), represented by floating-point numbers.

## 2.3 Data Quality Issues

Flawed data invalidates downstream analytical pipelines (\*Garbage In, Garbage Out\*). Redman (2004) estimated poor data quality costs organizations 10% ÷ 20% of total revenue.

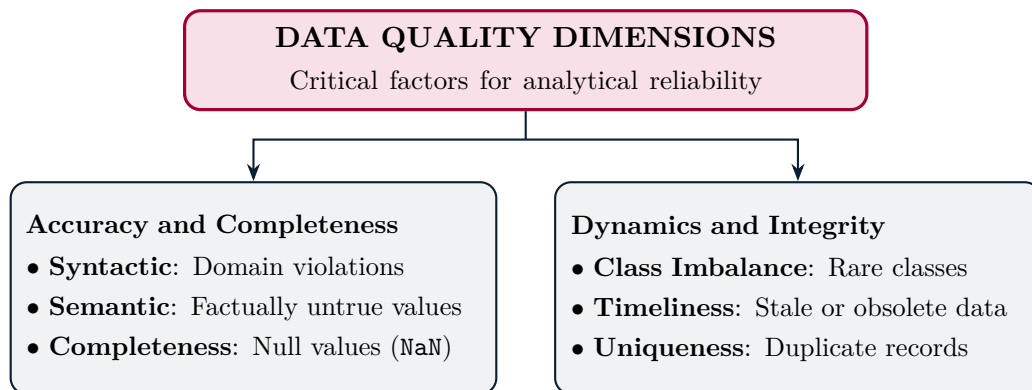


Figure 2.2: Core dimensions of data quality.

## 2.4 Univariate Distribution Inspection

### 2.4.1 Skewness and Multimodality

The shape of continuous probability density functions reveals underlying mechanisms:

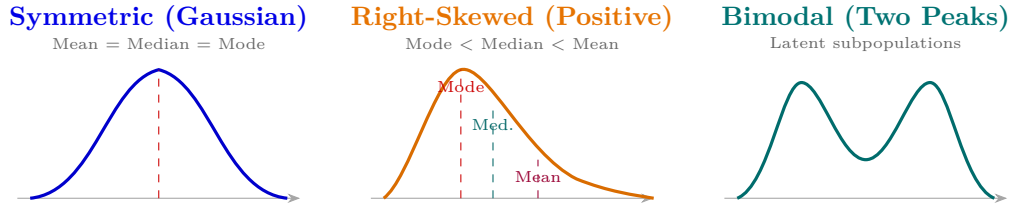


Figure 2.3: Univariate distribution profiles: symmetry, positive skewness, and multimodality.

- **Symmetric Distribution:** Reflectional symmetry about the center: Mean = Median = Mode.
- **Positively Skewed (Right-Skewed):** Long tail towards higher values (e.g., individual wealth):

$$\text{Mode} < \text{Median} < \text{Mean} \quad (2.2)$$

- **Negatively Skewed (Left-Skewed):** Tail stretching towards small values:

$$\text{Mean} < \text{Median} < \text{Mode} \quad (2.3)$$

- **Bimodal / Multimodal Distribution:** Signals overlapping subpopulations (e.g., adult height without conditioning on biological sex).

#### 2.4.2 Visual Tools: Bar Charts vs Histograms

- **Bar Chart:** Plots discrete categories with visible spacing between bars.
- **Histogram:** Partitions continuous domains into contiguous intervals (\*bins\*); rectangle area is proportional to empirical sample counts.

**Bin Sizing and Sturges' Rule:** Under approximately normal distributions, Sturges' rule determines optimal bin counts  $k$ :

$$k = \lceil \log_2(n) + 1 \rceil \quad (2.4)$$

## 2.5 Measures of Central Tendency and Dispersion

### 2.5.1 Central Tendency

- **Sample Mean:**

$$\bar{x} = \frac{1}{m} \sum_{i=1}^m x_i \quad (2.5)$$

Highly vulnerable to severe outliers.

- **Median:** Positional center in sorted arrays:

$$\text{median} = \begin{cases} x_{(\frac{m+1}{2})} & \text{if } m \text{ is odd} \\ \frac{x_{(\frac{m}{2})} + x_{(\frac{m}{2}+1)}}{2} & \text{if } m \text{ is even} \end{cases} \quad (2.6)$$

A robust indicator resistant to extreme values.

- **Mode:** Most frequently observed value; uniquely defined across nominal attributes.
- **Trimmed Mean:** Evaluated by dropping a proportion  $\alpha$  of extremes.

### 2.5.2 Measures of Dispersion

- **Range:**  $\text{Range} = x_{\max} - x_{\min}$ .
- **Sample Variance ( $s^2$ ):**

$$s^2 = \frac{1}{m-1} \sum_{i=1}^m (x_i - \bar{x})^2 \quad (2.7)$$

using Bessel's correction ( $m-1$ ) for unbiased population estimation.

- **Sample Standard Deviation ( $s$ ):**  $s = \sqrt{s^2}$ .

#### Robust Dispersion Indicators:

- **Absolute Average Deviation (AAD):**

$$\text{AAD}(x) = \frac{1}{m} \sum_{i=1}^m |x_i - \bar{x}| \quad (2.8)$$

- **Median Absolute Deviation (MAD):**

$$\text{MAD}(x) = \text{median} \left( \left\{ |x_i - \text{median}(x)| \right\}_{i=1}^m \right) \quad (2.9)$$

## 2.6 Five-Number Summary, Quartiles, and Box Plots

### 2.6.1 The Five-Number Summary

The **Five-Number Summary** characterizes distributions using positional order statistics:

$$\{x_{\min}, Q_1, \text{Median } (Q_2), Q_3, x_{\max}\} \quad (2.10)$$

- $Q_1$  (First Quartile): 25th percentile.
- $Q_2$  (Median): 50th percentile.
- $Q_3$  (Third Quartile): 75th percentile.
- **Interquartile Range (IQR)**:  $\text{IQR} = Q_3 - Q_1$ .

### 2.6.2 Box Plot Anatomy

The **Box Plot** maps the five-number summary geometrically:

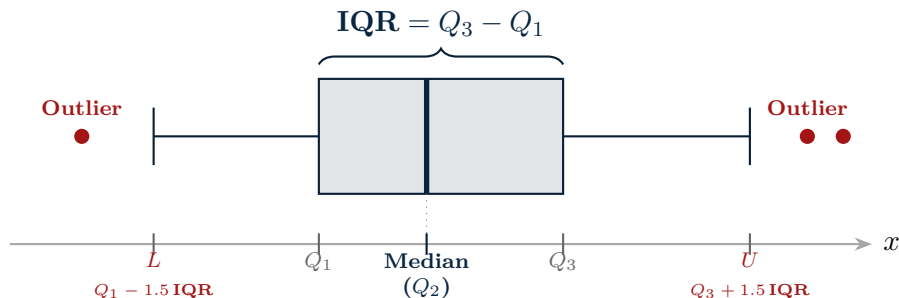


Figure 2.4: Box Plot anatomy with Tukey fences for univariate outlier demarcation.

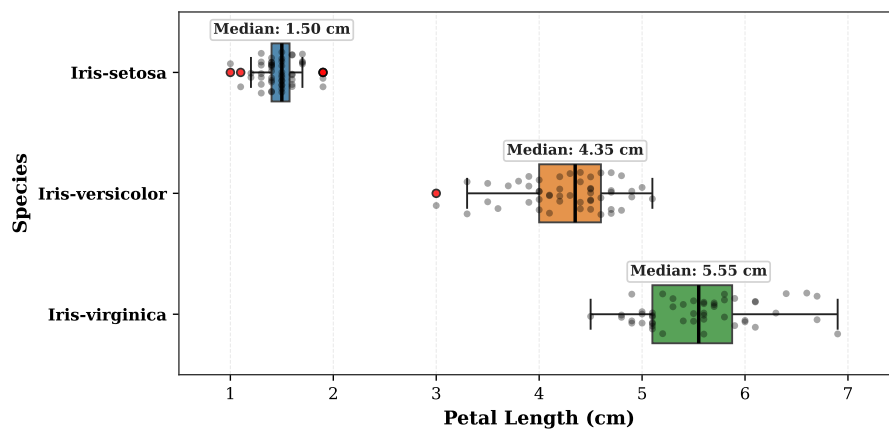


Figure 2.5: Empirical conditional box plots on the Iris dataset: petal length distributions partitioned by species.

**Limitations of Box Plots Relative to Histograms:** Box plots cannot detect multimodality. Identical five-number summaries can emerge from both bimodal and unimodal distributions; histograms remain necessary to reveal intermediate concavities.

## Chapter 3

# Data Understanding II – Multivariate Analysis, Correlation, and Outlier Detection

Multivariate analysis studies simultaneous relationships among two or more attributes within a dataset, overcoming the limitations of isolated univariate descriptions. In this chapter, we explore multidimensional visualization techniques, formal indices for linear and non-linear association and correlation, methodological criteria for outlier diagnosis, and an operational diagnostic checklist to conclude the Data Understanding phase.

The case study used as our benchmark is the famous **Iris** dataset (introduced by botanist Edgar Anderson and popularized by Ronald Fisher in 1936), composed of 150 observations evenly distributed across 3 balanced classes of 50 samples each (*Iris setosa*, *Iris versicolor*, *Iris virginica*) and described by 4 continuous features measured in centimeters:

- Sepal length (*Sepal Length*)
- Sepal width (*Sepal Width*)
- Petal length (*Petal Length*)
- Petal width (*Petal Width*)

### 3.1 Multidimensional Visualization Techniques

When exploratory analysis extends to bivariate and multivariate configurations, graphical representations must provide immediate visual insight

into geometric class separability, natural cluster formations, and structural anomalies.

### 3.1.1 Scatter Plot and Scatter Matrix

The **Scatter Plot** pairs two continuous attributes  $(X, Y)$ , projecting observations as points with Cartesian coordinates  $(x_i, y_i) \in \mathbb{R}^2$ .

- **Key Capabilities:** Visually highlights clusters, direct or inverse correlation trends, and bivariate outliers that remain undetectable in univariate histograms.
- **Color Encoding:** By assigning distinct colors or markers according to class labels, practitioners can assess the geometric separability induced by the feature pair.
- **Evidence from Iris:** Crossing *Petal Length* and *Petal Width* reveals sharp class separation: *Setosa* forms an isolated compact cluster, whereas *Versicolor* and *Virginica* exhibit only modest boundary overlap (Figure 3.1). In contrast, the plane formed by *Sepal Length* and *Sepal Width* shows substantial inter-class confusion between *Versicolor* and *Virginica*.

When the number of numeric features  $p$  is moderate ( $3 \leq p \leq 10$ ), a **Scatter Matrix (Scatter Plot Matrix - SPLOM)** is employed: a symmetric  $p \times p$  grid containing all  $\frac{p(p-1)}{2}$  distinct pairwise scatter plots. Univariate histograms or kernel density estimates (KDE) are typically placed along the main diagonal, enabling simultaneous inspection of univariate distributions and pairwise correlations (Figure 3.2).

The **Three-Dimensional (3D) Scatter Plot** extends Cartesian projection to three coordinates simultaneously  $(X_1, X_2, X_3)$ . It allows identifying 3D outliers that appear completely normal in 2D projections, though planar displays suffer from visual occlusion and viewpoint dependence.

### 3.1.2 Visualizing High-Dimensional Spaces

In spaces where dimension count exceeds  $p > 3$ , standard orthogonal Cartesian projections fail to provide comprehensive views. Alternative coordinate geometries are required.

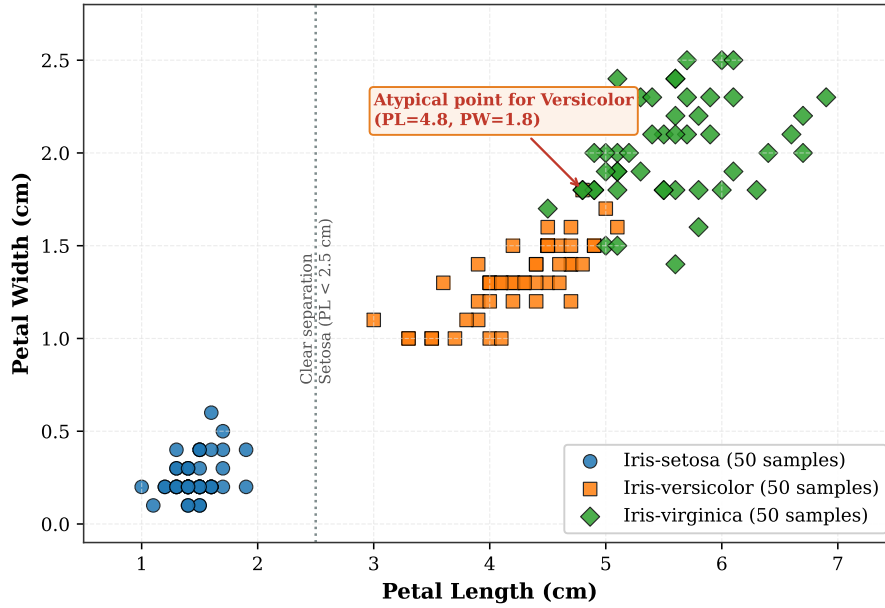


Figure 3.1: Bivariate scatter plots on the Iris dataset: comparing sepal and petal class separability.

### Parallel Coordinates

In parallel coordinates, coordinate axes are not mutually orthogonal; instead, they are drawn as  $p$  equidistant, parallel vertical lines, each calibrated to the value range of its corresponding feature.

- **Geometric Mapping:** A  $p$ -dimensional observation  $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \in \mathbb{R}^p$  is represented not as a point, but as a continuous **polyline** crossing all parallel axes, intersecting the  $j$ -th vertical axis at height  $x_{ij}$ .
- **Diagnostic Traits:** Records sharing identical class labels tend to form compact bundles of polylines displaying coherent trajectories. An isolated polyline with an anomalous trajectory immediately flags a potential multivariate outlier.
- **Axis Ordering:** Visual legibility depends strongly on horizontal axis ordering. Placing strongly correlated attributes on adjacent axes reveals parallel bundles (positive correlation) or pencil-shaped X-crossings (negative correlation) (Figure 3.3).



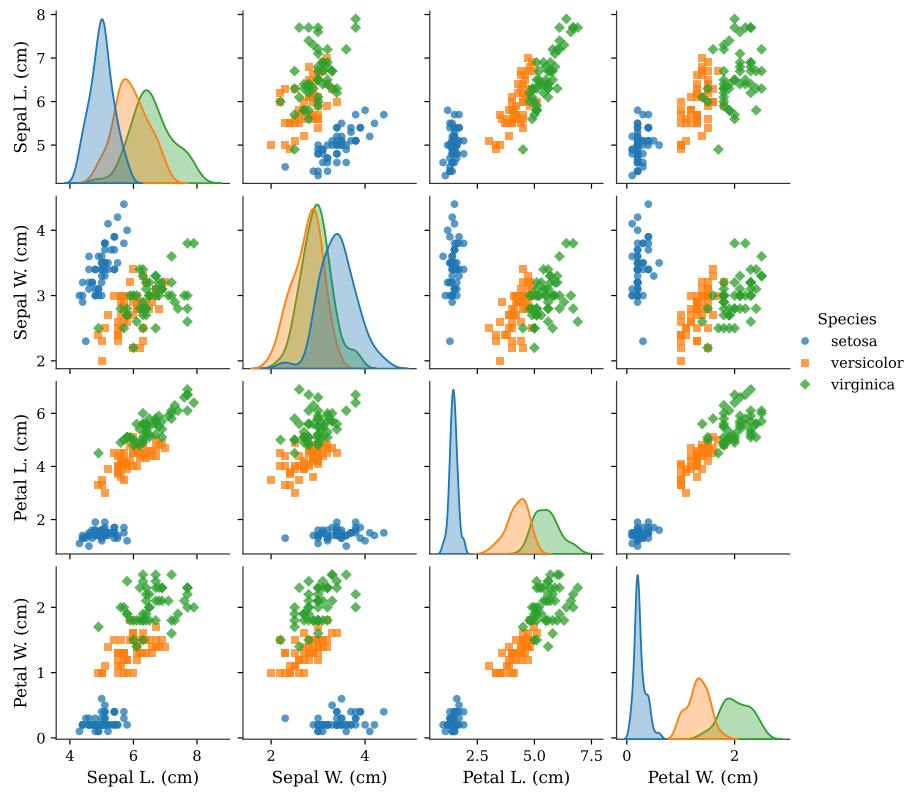


Figure 3.2: Complete Scatter Plot Matrix (SPLOM) across the four attributes of the Iris dataset.

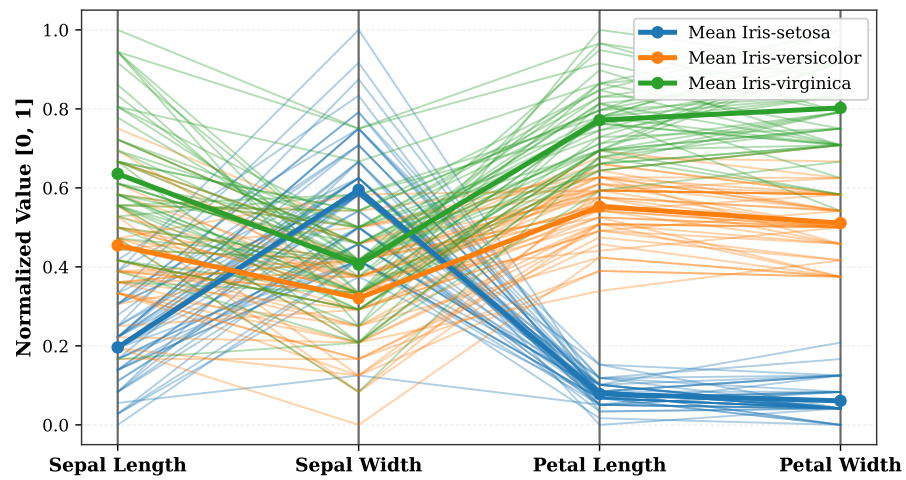


Figure 3.3: Parallel coordinates for the Iris dataset with class-stratified trajectories.

### Radar Plot (Spider or Star Chart)

The **Radar Plot** shares the structural principle of parallel coordinates, but dimensional axes are arranged **radially emanating from a shared central pole**, separated by equi-spaced angles:

$$\theta_j = \frac{2\pi(j-1)}{p}, \quad j = 1, \dots, p \quad (3.1)$$

For each data record, feature values determine the vertex distance along the corresponding radial spoke; connecting consecutive vertices produces a closed polygon. When radar plots are displayed as small individual multiples for each observation across a grid, they are called **Star Plots**, facilitating rapid geometric comparisons across multivariate instances (Figure 3.4).

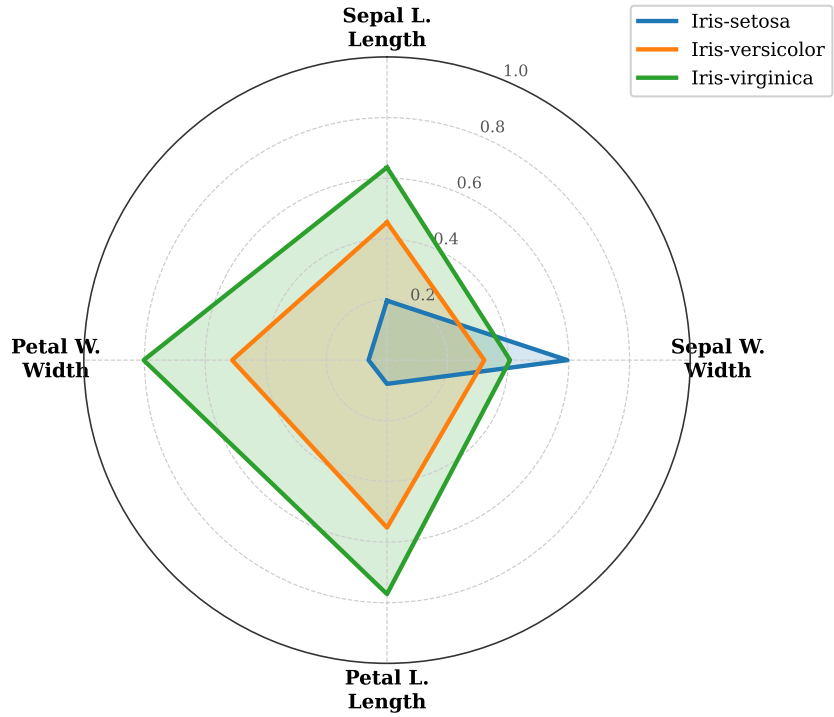


Figure 3.4: Class-average profiles for the Iris dataset rendered via Radar Plot.

## 3.2 Bivariate Dependence Analysis: Covariance and Correlation

Quantitative dependence analysis investigates the strength and direction with which two random variables co-vary.

### 3.2.1 From Variance to Covariance

While variance measures the dispersion of a single variable around its own expectation, **Sample Covariance** measures the joint tendency of two numerical variables  $X$  and  $Y$  to deviate synchronously from their sample means:

$$\text{Cov}(X, Y) = s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \quad (3.2)$$

In the special case where  $X = Y$ , covariance equals sample variance:

$$\text{Cov}(X, X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 = s_x^2 \quad (3.3)$$

### Covariance Matrix

For a set of  $p$  continuous attributes, joint variability is completely captured by the **Covariance Matrix**  $V \in \mathbb{R}^{p \times p}$ :

$$V = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_p) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) & \cdots & \text{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_p, X_1) & \text{Cov}(X_p, X_2) & \cdots & \text{Cov}(X_p, X_p) \end{bmatrix} \quad (3.4)$$

- Diagonal entries  $V_{jj} = \text{Var}(X_j) = s_j^2$  represent marginal sample variances.
- Off-diagonal entries  $V_{jk} = \text{Cov}(X_j, X_k)$  represent pairwise covariances.
- Since  $\text{Cov}(X_j, X_k) = \text{Cov}(X_k, X_j)$ , matrix  $V$  is **symmetric** ( $V = V^T$ ) and positive semi-definite ( $V \succeq 0$ ).

### Geometric Interpretation of Deviation Products

The sign of sample covariance is governed by the quadrants formed around the bivariate mean  $(\bar{x}, \bar{y})$  in Cartesian space:

- Points in Quadrants I and III contribute positive products (concordant deviations)  $\implies \text{Cov}(X, Y) > 0$ .

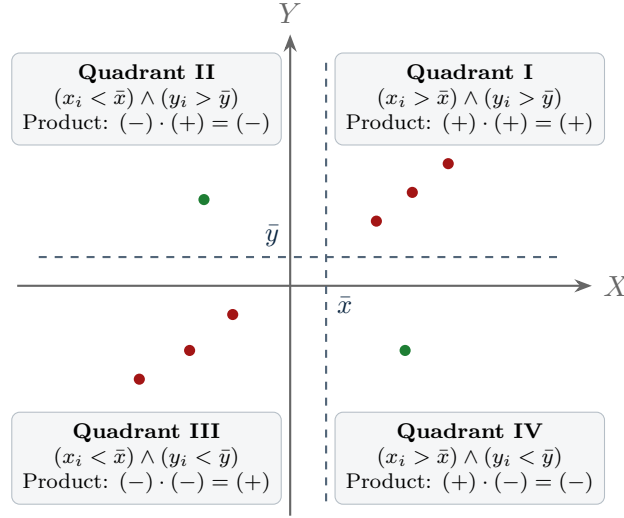


Figure 3.5: Geometric decomposition of bivariate deviation products across four mean-centered quadrants.

- Points in Quadrants II and IV contribute negative products (discordant deviations)  $\implies \text{Cov}(X, Y) < 0$ .
- Uniformly dispersed observations cancel out across quadrants, yielding  $\text{Cov}(X, Y) \approx 0$ .

### Structural Limitation: Scale Dependence

Covariance lacks scale invariance: its magnitude scales linearly with measurement units. Converting income  $X$  from euros to thousands of euros ( $X' = X/1000$ ) shrinks covariance by a factor of 1000, although the underlying statistical link remains identical. Hence, covariance cannot directly compare pairs with distinct units or scales.

### 3.2.2 Pearson Linear Correlation Coefficient

The **Pearson Linear Correlation Coefficient** ( $r_{xy}$ ) standardizes covariance by dividing by the product of both sample standard deviations:

$$r_{xy} = \frac{\text{Cov}(X, Y)}{s_x s_y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (3.5)$$

### Mathematical Properties of $r_{xy}$

1. **Bounded Range:** By the Cauchy-Schwarz inequality on centered vectors, the coefficient is strictly bounded within:

$$-1 \leq r_{xy} \leq +1 \quad (3.6)$$

2. **Scale Invariance:** It is a dimensionless index invariant under positive affine transformations ( $X' = aX + b, Y' = cY + d$  with  $a, c > 0$ ).

3. **Perfect Linear Relationship:**

- $r_{xy} = +1$ : perfect positive linear correlation; all points lie exactly on a line with positive slope ( $y = ax + b, a > 0$ ).
- $r_{xy} = -1$ : perfect negative linear correlation ( $a < 0$ ).

4. **Absence of Linear Relationship:** A value of  $r_{xy} \approx 0$  signifies **absence of linear association**, but does not imply statistical independence. Non-linear deterministic functions (e.g.,  $Y = X^2$  with  $X$  symmetrically distributed around zero) yield  $r_{xy} = 0$ , despite full underlying determinism.

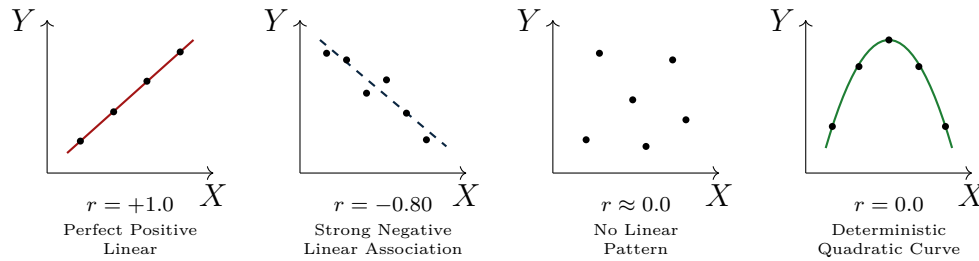


Figure 3.6: Archetypal scatter patterns and their corresponding Pearson correlation values.

### 3.2.3 Causal Fallacy and Confounding Variables

A cornerstone of data science states that **correlation does not imply causation**. A high observed  $|r_{xy}|$  may arise from three distinct phenomena:

1.  $X$  is a direct causal driver of  $Y$ .
2.  $Y$  is a direct causal driver of  $X$ .

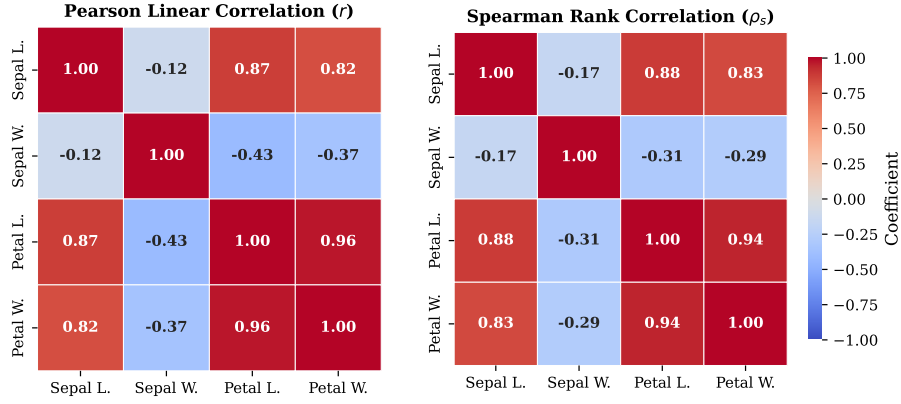


Figure 3.7: Correlation heatmaps for the overall Iris dataset and stratified by species.

3.  $X$  and  $Y$  are simultaneously driven by an unobserved or latent third factor, termed a **confounding variable**.

#### Classic Example: Ice Cream Sales and Drowning Incidents

During summer months, a high positive correlation ( $r > 0.85$ ) is documented between ice cream sales ( $X$ ) and drowning casualties ( $Y$ ). Inferring that ice cream consumption increases drowning risk represents a classic causal fallacy: both variables are driven by an underlying exogenous confounder—high atmospheric temperature (*hot weather*)—which spurs both outdoor swimming and cold dessert purchases.

### 3.2.4 Spearman Rank Correlation Coefficient

When variables violate bivariate normality assumptions or exhibit non-linear yet strictly monotonic relationships, the **Spearman Rank Correlation Coefficient** ( $\rho$ ) is preferred.

The method replaces continuous values  $x_i$  and  $y_i$  with their corresponding ordinal ranks  $r(x_i), r(y_i) \in \{1, \dots, n\}$ . In the absence of ties, the explicit formulation simplifies to:

$$\rho = 1 - \frac{6 \sum_{i=1}^n (r(x_i) - r(y_i))^2}{n(n^2 - 1)} \quad (3.7)$$

- $\rho = +1$ : indicates a perfect monotonic increasing relationship (as  $X$  increases,  $Y$  strictly increases, even along exponential or logarithmic curves).

- $\rho = -1$ : indicates a perfect monotonic decreasing relationship.
- **Robustness**: Relying on ranks rather than raw metric differences renders Spearman significantly more resistant to extreme outliers in distribution tails.

### 3.3 Outlier Taxonomy and Detection Methodologies

An **outlier** is defined as an observation that deviates so substantially from the remaining data as to arouse suspicion that it was generated by a different mechanism or contaminated by measurement error.

#### 3.3.1 The Dual Nature of Outliers: Noise vs. Primary Target

- **Outliers as Noise**: The anomaly stems from data entry typos, temporary sensor glitches, or spurious noise. In this role, outliers distort parameter estimation and should be identified, corrected, or removed during data preprocessing.
- **Outliers as the Primary Target (Analysis Goal)**: The explicit mission of the analysis is to uncover rare, high-impact events carrying crucial domain insights:
  - Intrusion detection in computer network security.
  - Financial fraud detection and anti-money laundering monitoring.
  - Predictive maintenance through anomalous vibration patterns in industrial turbines.

#### 3.3.2 Outlier Identification Methodologies

Outlier identification methodologies can be systematically categorized into univariate and multidimensional approaches:

- **Categorical Attributes**: An anomalous observation belongs to a rare category occurring with near-zero empirical frequency ( $< 0.1\%$ ).
- **Univariate Numeric Attributes**:
  - *Tukey IQR Rule*: Values outside the interval  $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$ .

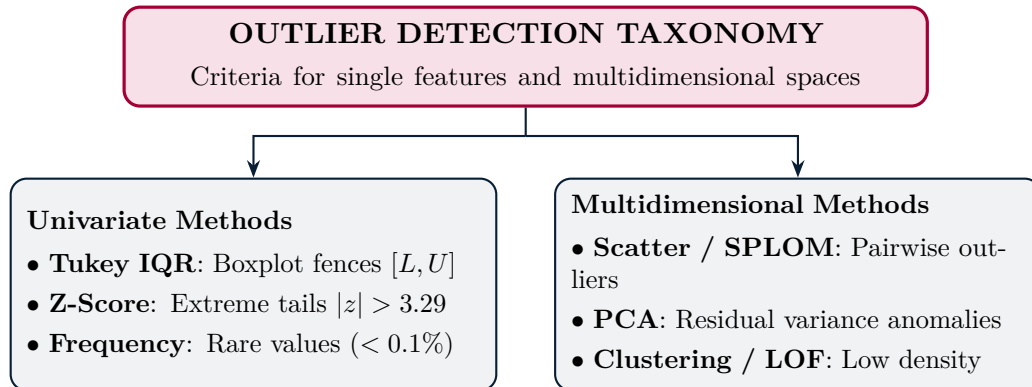


Figure 3.8: Taxonomy of outlier detection methodologies.

- *Z-Score Rule*: Standardized values with  $|z| > 3.29$  (representing the outermost 0.1% tails of a Gaussian distribution).
- **Multidimensional Spaces**:
  - *Bivariate Scatter Plots*: Visual identification of points detached from dense clusters.
  - *PCA Projections*: Observations projecting as outliers along secondary principal components.
  - *Clustering and Local Outlier Factor (LOF)*: Isolating samples residing in low-density neighborhoods relative to their  $k$ -nearest neighbors.

## 3.4 Introduction to Missing Values

A data instance contains **Missing Values** when no observation is recorded for one or more features (explicitly encoded via symbols like NaN, NULL, ? or implicitly hidden under sentinel conventions such as 999 or 0).

### 3.4.1 Root Causes of Missingness

1. **Voluntary Non-Response**: Respondents decline to disclose sensitive personal data (e.g., annual income, political affiliations, medical history).
2. **Conceptual Inapplicability**: The attribute is logically inapplicable for certain subpopulations (e.g., pregnancy counts for male subjects, or employment tenure for school-age minors).



3. **Hardware Failures or Dropped Connections:** Telemetry sensors freezing or disconnecting, producing null readings or reporting continuous dummy constants (e.g., a frozen wind sensor persistently reporting 0 m/s).

### 3.4.2 Operational Data Understanding Checklist

Before proceeding to data cleaning and preparation, practitioners must systematically verify the following diagnostic checklist:

| Diagnostic Check                    | Operational Verification Procedure                                                  |
|-------------------------------------|-------------------------------------------------------------------------------------|
| <b>Syntactic Integrity</b>          | Validate data types, string/number formats, and domain constraints.                 |
| <b>Outlier Inspection</b>           | Inspect extreme values using univariate Box-plots and bivariate Scatter Plots.      |
| <b>Missing Value Audit</b>          | Quantify null percentages and detect implicit sentinel values (0, -1, 999).         |
| <b>Association &amp; Redundancy</b> | Compute Pearson and Spearman matrices to identify collinear or redundant features.  |
| <b>Distributional Assumptions</b>   | Check symmetry, normality, and variance stability prior to feature transformations. |

Table 3.1: Operational diagnostic checklist for completing the Data Understanding phase.

## Chapter 4

# Data Preparation & Cleaning – Reduction, Discretization, and Transformation

While Data Understanding focuses on diagnosing anomalies and comprehending latent structures within a dataset, **Data Preparation** represents the operational, interventional component of the KDD lifecycle. During this phase, raw records are cleansed, compressed, scaled, and re-encoded to satisfy the algebraic and computational requirements of machine learning and data mining algorithms.

### 4.1 Distinction Between Data Understanding and Data Preparation

The transition from diagnostic exploration to operational data engineering is captured by the clear mapping between diagnostic findings and corresponding corrective actions:

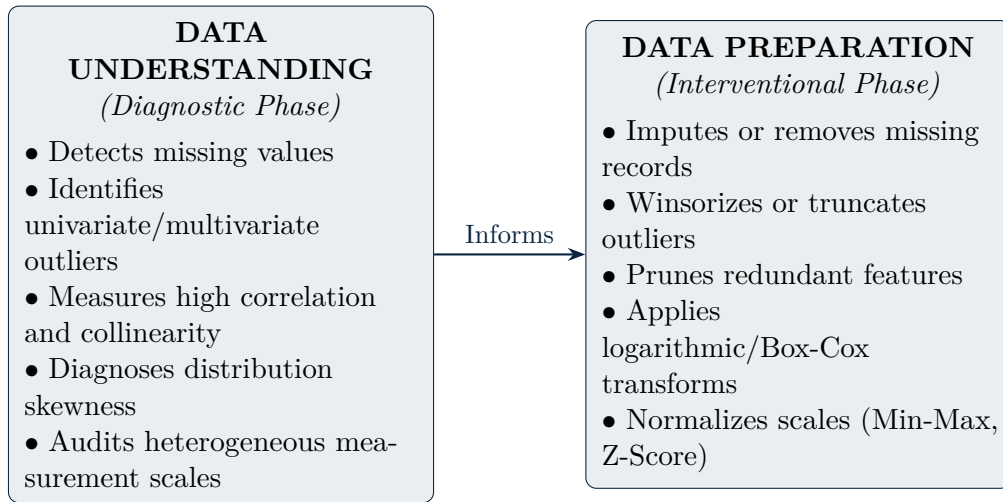


Figure 4.1: Sequential transition from Data Understanding (diagnosis) to Data Preparation (intervention).

## 4.2 Aggregation and Sampling

Reducing dataset volume can be achieved either by combining related observations or through statistical sampling of representative subsets.

### 4.2.1 Data Aggregation

Data aggregation merges two or more records or features into a single synthesized summary.

- **Core Objectives:**
  1. *Dimensional Reduction*: Reduces the total record count or feature cardinality, lowering computational complexity.
  2. *Scale Transformation*: Adjusts analysis to an appropriate conceptual level of granularity (e.g., aggregating hourly transactional logs into weekly sums, or regionalizing individual point-of-sale terminals).
  3. *Variance Stabilization*: Aggregated aggregates average out high-frequency noise, yielding lower-variance metrics and more robust statistical trends.
- **Case Study: Australian Precipitation (1982–1993)**: In historical Australian rainfall data, the standard deviation of monthly average

rainfall exhibits high dispersion with long tails reaching up to 15 cm. Conversely, when aggregating into annual averages, the standard deviation tightly clusters around low values with substantially reduced spread (Figure 4.2). To model seasonal agricultural patterns, the disaggregated monthly scale is essential, whereas comparing macroscopic regional climates benefits from the stable annual scale.

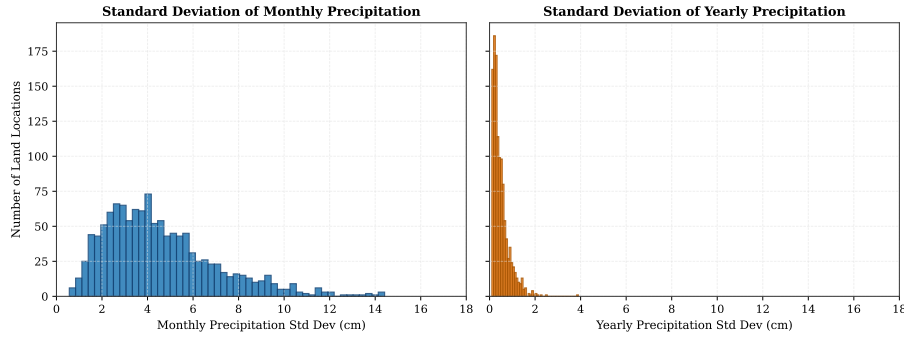


Figure 4.2: Dispersion comparison between monthly precipitation and aggregated annual precipitation in Australia.

## 4.2.2 Sampling Methodologies

Sampling selects a representative subset of size  $n$  from a large population of size  $N$  ( $n \ll N$ ). A sample is deemed **representative** if it preserves the underlying statistical properties and distributional profile of the parent population.

1. **Simple Random Sampling:** Every instance in the parent dataset has identical a priori inclusion probability:

$$P(x_i) = \frac{1}{N} \quad (4.1)$$

- *Without Replacement:* Extracted items are removed from the candidate pool and cannot be drawn again.
  - *With Replacement:* Extracted items remain in the pool and may appear multiple times in the sample.
2. **Stratified Sampling:** The dataset is partitioned into mutually disjoint, homogeneous subsets called *strata* (e.g., grouped by target class label or numeric percentiles). Independent random samples are extracted from each stratum proportional to its relative population size, preventing rare classes from being erased.

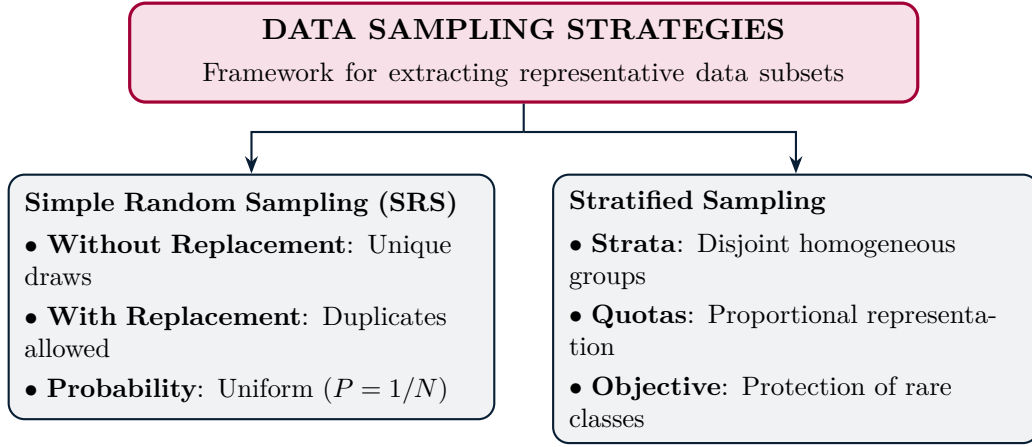


Figure 4.3: Taxonomy of data sampling strategies in data mining.

Selecting an optimal sample size involves a trade-off between computational processing cost and statistical fidelity, as depicted in Figure 4.4.

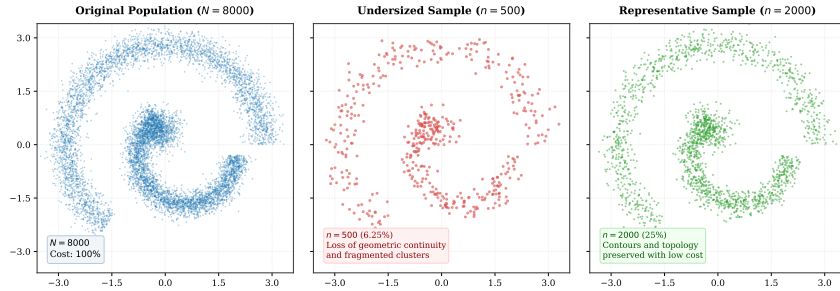


Figure 4.4: Trade-off curve between sample size and empirical parameter estimation error.

## 4.3 Dimensionality Reduction

High feature counts introduce significant computational and geometric bottlenecks.

### 4.3.1 The Curse of Dimensionality

As feature dimensionality  $p$  grows, the geometric volume of the embedding space  $\mathbb{R}^p$  expands exponentially relative to the available sample count. Key consequences include:

1. **Extreme Sparsity:** Data observations become isolated in high-dimensional space; local point density vanishes towards zero, leaving the feature space virtually empty.
2. **Distance Concentration:** As  $p \rightarrow \infty$ , the contrast between the maximum Euclidean distance and minimum Euclidean distance to any query point collapses:

$$\lim_{p \rightarrow \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} \rightarrow 0 \quad (4.2)$$

This severely impairs distance-based algorithms ( $k$ -NN, K-Means clustering, local density estimators).

3. **Covariance Instability:** Reliably estimating a  $p \times p$  sample covariance matrix requires sample sizes that scale exponentially with  $p$  to avoid rank deficiency and ill-conditioning.

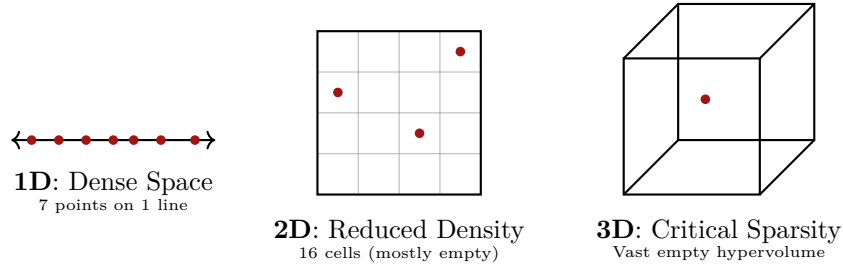


Figure 4.5: Visualizing exponential geometric dispersion of data samples with increasing dimensions.

### 4.3.2 Feature Selection

**Feature Selection** isolates an optimal subset of original attributes by discarding irrelevant (uninformative) and redundant (duplicated) features:

### 4.3.3 Feature Creation and Projection

- **Feature Construction:** Formulates novel descriptive attributes by combining raw features using domain knowledge (e.g., calculating density as mass/volume or productivity as completed/scheduled hours).

| Paradigm        | Selection Mechanism                                                                                                                | Trade-offs                                                                             |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Filter</b>   | Ranks features a priori using univariate statistical metrics (ANOVA, Pearson $r$ , $\chi^2$ ), independent of learning models.     | Blazing fast; ignores multivariate feature interactions.                               |
| <b>Wrapper</b>  | Uses the target machine learning model as an evaluation oracle across candidate subsets (Forward Selection, Backward Elimination). | High predictive accuracy; computationally heavy ( $2^p$ potential subset evaluations). |
| <b>Embedded</b> | Selection occurs natively during algorithm optimization (e.g., decision tree branch splits, Lasso $L_1$ penalty regularization).   | Excellent balance of computational efficiency and predictive accuracy.                 |

Table 4.1: Comparative breakdown of Filter, Wrapper, and Embedded feature selection approaches.

- **Feature Projection:** Algebraically projects original vectors from  $\mathbb{R}^p$  into a compact subspace  $\mathbb{R}^k$  ( $k \ll p$ ), preserving maximal variance or class separation (PCA, SVD, Linear Discriminant Analysis - LDA, Autoencoders).

## 4.4 Principal Component Analysis (PCA)

**Principal Component Analysis (PCA)** is a linear, unsupervised technique that maps an original set of correlated features into a new coordinate system of mutually orthogonal (uncorrelated) variables, ordered in descending fashion by explained variance.

### 4.4.1 Step-by-Step PCA Mathematical Derivation

The formal mathematical workflow of PCA proceeds across five structured steps:

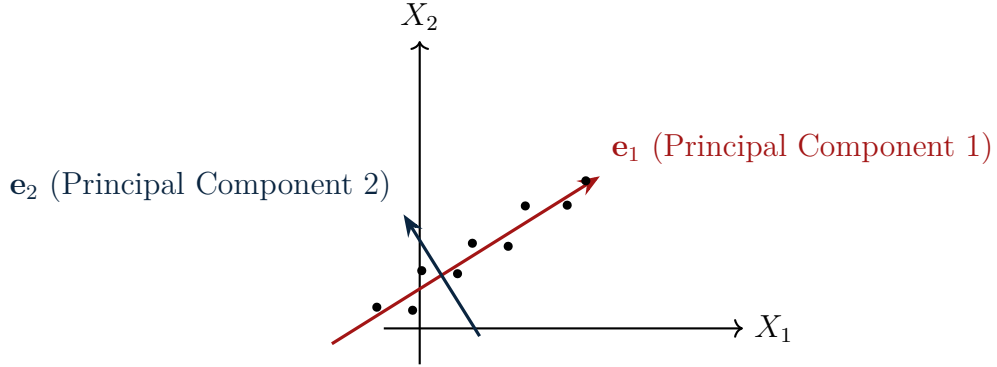


Figure 4.6: Alignment of orthogonal principal component axes along directions of maximal data variance.

### Step 1: Centering and Standardizing the Data Matrix

Given a data matrix  $D \in \mathbb{R}^{n \times p}$ , compute the sample mean for each feature column:

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n D_{ij} \quad (4.3)$$

Construct the **centered matrix**  $C \in \mathbb{R}^{n \times p}$  by subtracting feature means from each observation:

$$C_{ij} = D_{ij} - \bar{x}_j \quad (4.4)$$

If attributes exhibit dissimilar scales or measurement units, construct the **standardized matrix**  $Z \in \mathbb{R}^{n \times p}$  by dividing centered residuals by feature standard deviations  $s_j$ :

$$Z_{ij} = \frac{D_{ij} - \bar{x}_j}{s_j} \quad (4.5)$$

### Step 2: Sample Covariance Matrix Computation

Compute the empirical covariance matrix  $V \in \mathbb{R}^{p \times p}$  via matrix multiplication:

$$V = \frac{1}{n-1} C^T C \quad (4.6)$$

### Step 3: Spectral Decomposition (Eigenvalues and Eigenvectors)

Solve the characteristic eigenvalue equation for symmetric matrix  $V$ :

$$V \mathbf{e}_j = \lambda_j \mathbf{e}_j, \quad j = 1, \dots, p \quad (4.7)$$



- Each normalized unit-length **eigenvector**  $\|\mathbf{e}_j\|_2 = 1$  establishes the spatial direction of the  $j$ -th principal axis. Because  $V$  is symmetric, eigenvectors belonging to distinct eigenvalues are mutually orthogonal ( $\mathbf{e}_j^T \mathbf{e}_k = 0$  for  $j \neq k$ ).
- Each corresponding **eigenvalue**  $\lambda_j \geq 0$  quantifies the exact portion of sample variance explained along the axis  $\mathbf{e}_j$ .

#### Step 4: Sorting and Component Retention

Sort eigenvalues and associated eigenvectors in descending numerical order:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0 \quad (4.8)$$

The proportion of total variance explained by the  $j$ -th component equals:

$$\text{Explained Variance (\%)} = \frac{\lambda_j}{\sum_{m=1}^p \lambda_m} \times 100 \quad (4.9)$$

Retain the top  $k$  components ( $k < p$ ) satisfying an aggregate cumulative variance threshold (e.g., 85% or 95%), discarding residual trailing components (Figure 4.7). Construct the projection matrix  $F \in \mathbb{R}^{p \times k}$ :

$$F = [\mathbf{e}_1 \quad \mathbf{e}_2 \quad \dots \quad \mathbf{e}_k] \quad (4.10)$$

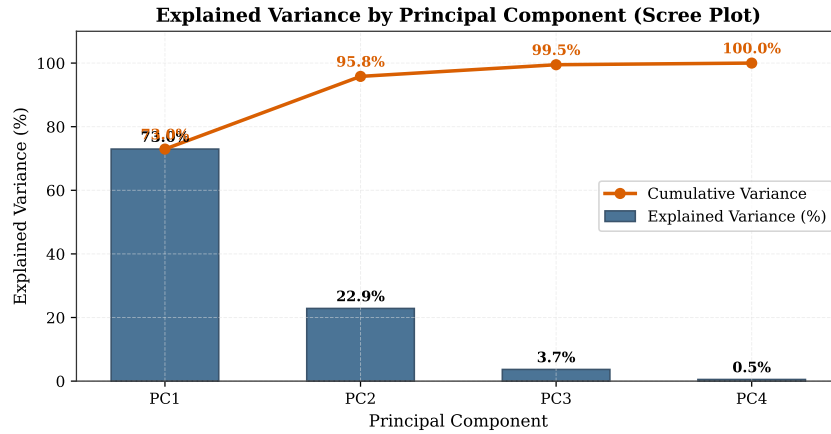


Figure 4.7: Eigenvalue scree plot and cumulative explained variance curve.

### Step 5: Subspace Projection

Project the centered dataset onto the reduced  $k$ -dimensional coordinate system:

$$Z = CF \quad (4.11)$$

Figure 4.8 illustrates the 2D projection of the Iris dataset onto its first two principal components (PC1 and PC2), which capture over 95% of total dataset variance while achieving distinct species clustering.

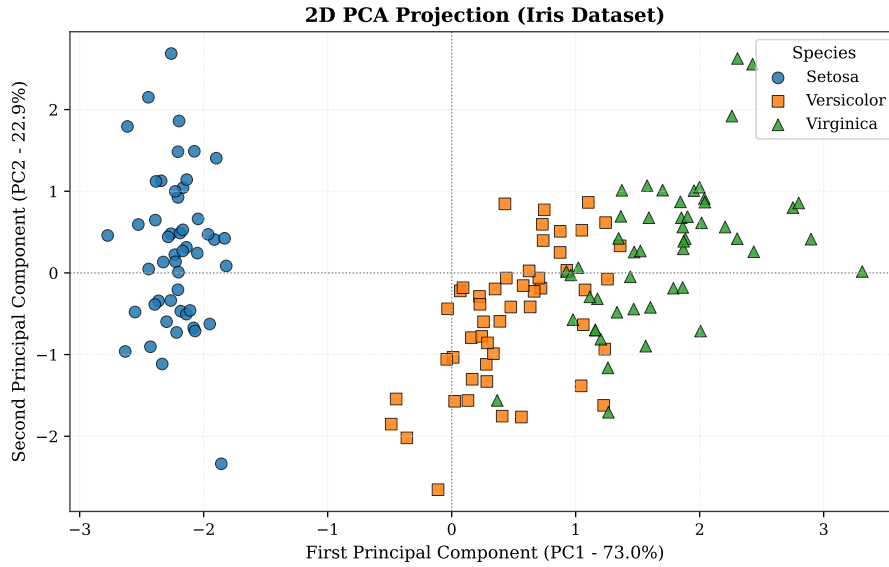


Figure 4.8: Two-dimensional projection of Iris samples onto the top two principal components.

## 4.5 Handling Missing Values

Operational strategies for incomplete records include:

1. **Listwise Deletion:** Discarding all records containing at least one missing attribute. Safe only when missing frequency is trivial ( $< 3-5\%$ ) and mechanism is strictly *Missing Completely at Random (MCAR)*; otherwise, severe sampling bias ensues.
2. **Central Tendency Imputation:**
  - *Mean:* For symmetric numeric variables lacking outliers.

- *Median*: For skewed numeric features with heavy tails or outliers.
  - *Mode*: For categorical attributes.
  - *Caveat*: Artificially deflates feature variance and weakens inter-attribute correlation structures.
3. **Conditional Segmented Imputation**: Grouping instances into clusters or classes based on complete features, then imputing missing values using the mean or median calculated within that homogeneous segment.
  4. **Model-Based Predictive Imputation**: Fitting a supervised model ( $k$ -NN, linear regression, Random Forest) where the missing feature acts as the target variable and complete attributes serve as predictors.

## 4.6 Discretization and Binarization Techniques

**Discretization** converts continuous quantitative variables into discrete ordinal attributes by partitioning the continuous support into a finite set of contiguous intervals (termed *bins*).

### 4.6.1 Unsupervised Discretization

#### 1. Natural Binning (Equal-Width Binning)

Partitions the range  $[x_{\min}, x_{\max}]$  into  $k$  equal-sized intervals of **constant width**:

$$\delta = \frac{x_{\max} - x_{\min}}{k} \quad (4.12)$$

The  $i$ -th bin ( $i = 0, \dots, k - 1$ ) spans:

$$I_i = [x_{\min} + i\delta, x_{\min} + (i + 1)\delta) \quad (4.13)$$

#### Numerical Example: 12 Beer Prices

Given  $N = 12$  beer prices spanning from  $x_{\min} = 100$  to  $x_{\max} = 160$ , choosing  $k = 4$  bins yields interval width  $\delta = \frac{160-100}{4} = 15$ . Intervals and counts:

- Bin 1:  $[100, 115) \rightarrow \{100, 110, 110\}$  (Count: 3)
- Bin 2:  $[115, 130) \rightarrow \{120, 120, 125\}$  (Count: 3)
- Bin 3:  $[130, 145) \rightarrow \{130, 130, 135, 140\}$  (Count: 4)
- Bin 4:  $[145, 160] \rightarrow \{150, 160\}$  (Count: 2)

## 2. Equal-Frequency Binning (Equal-Depth Binning)

Sorts the data ascendingly and allocates an equal number of records into each bin:

$$f = \frac{N}{k} \quad (4.14)$$

For the same sorted 12 prices  $\{100, 110, 110, 120, 120, 125, 130, 130, 135, 140, 150, 160\}$  with  $k = 4$ , target count is  $f = 12/4 = 3$ :

- Bin 1:  $\{100, 110, 110\}$
- Bin 2:  $\{120, 120, 125\}$
- Bin 3:  $\{130, 130, 135\}$
- Bin 4:  $\{140, 150, 160\}$

This balances density and dampens outlier skewness, though identical values straddling bin boundaries may be artificially split into adjacent categories.

### Formulas for Optimal Bin Count and Width

Beyond Sturges' formula, the base-10 logarithmic rule provides:

$$C = 1 + \frac{10}{3} \log_{10}(N) \quad (4.15)$$

To optimize bin width  $h$  accounting jointly for sample spread and size, **Scott's Rule (1979)** provides:

$$h = \frac{3.5 \times s}{\sqrt[3]{N}} \quad (4.16)$$

where  $s$  is the sample standard deviation and  $N$  is sample size.

## 4.6.2 Supervised Entropy-Based Discretization

When a target class label  $Y \in \{c_1, \dots, c_m\}$  is available, split thresholds are selected to maximize class homogeneity. The Shannon entropy of dataset  $S$  is:

$$\text{Entropy}(S) = - \sum_{j=1}^m p_j \log_2(p_j) \quad (4.17)$$

where  $p_j$  is the proportion of samples belonging to class  $c_j$ . A candidate threshold  $T$  partitioning  $S$  into  $S_1$  and  $S_2$  yields weighted entropy:

$$E(S, T) = \frac{|S_1|}{|S|} \text{Entropy}(S_1) + \frac{|S_2|}{|S|} \text{Entropy}(S_2) \quad (4.18)$$

The optimal split  $T^*$  maximizes information gain  $\text{Gain}(S, T) = \text{Entropy}(S) - E(S, T)$ .

### 4.6.3 Binarization (Encoding)

Converts categorical attributes with  $m$  distinct levels into binary indicators  $\{0, 1\}$ :

- **One-Hot Encoding (Asymmetric Binary Encoding)**: Allocates a dedicated indicator feature for each category level. Avoids imposing artificial order:

| Category | Integer Code | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|----------|--------------|-------|-------|-------|-------|-------|
| awful    | 0            | 1     | 0     | 0     | 0     | 0     |
| poor     | 1            | 0     | 1     | 0     | 0     | 0     |
| OK       | 2            | 0     | 0     | 1     | 0     | 0     |
| good     | 3            | 0     | 0     | 0     | 1     | 0     |
| great    | 4            | 0     | 0     | 0     | 0     | 1     |

Table 4.2: One-Hot encoding for an attribute with 5 distinct qualitative categories.

- **Compact Binary Encoding**: Encodes category levels into positional binary integers using  $n = \lceil \log_2(m) \rceil$  bits: *Drawback*: Induces synthetic

| Category | Integer Code | $x_1$ | $x_2$ | $x_3$ |
|----------|--------------|-------|-------|-------|
| awful    | 0            | 0     | 0     | 0     |
| poor     | 1            | 0     | 0     | 1     |
| OK       | 2            | 0     | 1     | 0     |
| good     | 3            | 0     | 1     | 1     |
| great    | 4            | 1     | 0     | 0     |

Table 4.3: Compact binary encoding on 3 bits for 5 categories.

correlations and ordering artifacts among bits that lack real-world domain validity.

## 4.7 Functional Attribute Transformations and Variance Stabilization

A **functional transformation** applies a strictly monotonic, continuous function  $T$  to every value of feature  $X$ :  $Y = T(X)$ .

- **Variance Stabilization:** Enforces constant error variance across attribute ranges (*homoscedasticity*).
- **Distribution Normalization:** Transforms heavily skewed distributions toward approximate Gaussianity.
- **Relationship Linearization:** Transforms curvilinear relationships into linear ones suitable for correlation and regression.

#### 4.7.1 Power and Logarithmic Transformation Family

The general parametric transformation is formulated as:

$$T_p(x) = \begin{cases} ax^p + b & \text{if } p \neq 0 \\ c \log(x) + d & \text{if } p = 0 \end{cases} \quad (4.19)$$

with  $x > 0$ .

- **Linear Transformation** ( $p = 1$ ): Alters only scale and origin without changing distribution shape (e.g., Fahrenheit to Celsius:  $a = 5/9, b = -160/9$ ).
- **Logarithmic Transformation** ( $p = 0$ ):  $T(x) = c \log(x) + d$ . Compresses extreme upper-tail values while stretching small values, effectively correcting severe right skewness.

| Bar | Beer  | Original Gain ( $X$ ) | Transformed Gain $\log_{10}(X)$ |
|-----|-------|-----------------------|---------------------------------|
| A   | Bud   | 20                    | 1.3010                          |
| A   | Becks | 10000                 | 4.0000                          |
| C   | Bud   | 300                   | 2.4771                          |
| D   | Bud   | 400                   | 2.6021                          |
| D   | Becks | 5                     | 0.6990                          |
| E   | Becks | 120                   | 2.0792                          |
| E   | Bud   | 120                   | 2.0792                          |
| F   | Bud   | 11000                 | 4.0414                          |
| G   | Bud   | 1300                  | 3.1139                          |
| H   | Bud   | 3200                  | 3.5051                          |
| H   | Becks | 1000                  | 3.0000                          |
| I   | Bud   | 135                   | 2.1303                          |

Table 4.4: Empirical sample of 12 commercial Gain records before and after  $\log_{10}$  transformation.

| Statistical Metric                 | Raw Scale ( $X$ ) | Logarithmic Scale ( $Y = \log_{10} X$ ) |
|------------------------------------|-------------------|-----------------------------------------|
| Mean                               | 2481.82           | 2.5956                                  |
| Standard Deviation                 | 4079.02           | 1.0651                                  |
| Minimum                            | 5                 | 0.6990                                  |
| 1 <sup>st</sup> Quartile ( $Q_1$ ) | 120               | 2.0792                                  |
| Median ( $Q_2$ )                   | 400               | 2.6021                                  |
| 3 <sup>rd</sup> Quartile ( $Q_3$ ) | 2250              | 3.3095                                  |
| Maximum                            | 11000             | 4.0414                                  |

Table 4.5: Descriptive metrics before and after logarithmic transformation.

In the raw scale, standard deviation (4079.02) dwarfs the mean (2481.82), heavily pulled by outliers at 10000 and 11000. Following  $\log_{10}$  scaling, mean (2.5956) and median (2.6021) align closely, restoring symmetry.

#### 4.7.2 Other Variance Stabilizing Transformations

- **Square Root** ( $p = 1/2$ ):  $T(x) = \sqrt{x}$ . Stabilizes variance for Poisson count data ( $\sigma^2 = \mu$ ).
- **Reciprocal** ( $p = -1$ ):  $T(x) = 1/x$ . Applied when variance increases superlinearly relative to the mean.

### 4.8 Normalization Algorithms

Normalization scales diverse numerical features onto a shared, comparable range, preventing features with large magnitudes from dominating distance computations in  $k$ -NN, K-Means, SVM, or artificial neural networks.

#### 4.8.1 Min-Max Normalization

Linearly transforms values from original range  $[\min_A, \max_A]$  into a defined target interval  $[\text{new\_min}_A, \text{new\_max}_A]$  (typically  $[0, 1]$ ):

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new\_max}_A - \text{new\_min}_A) + \text{new\_min}_A \quad (4.20)$$

Preserves exact linear proportions, but remains vulnerable to out-of-bounds future inputs and extreme outliers compressing standard values.

### 4.8.2 Z-Score Standardization

Shifts and scales values such that the transformed variable has **zero mean and unit variance**:

$$v' = \frac{v - \bar{x}_A}{s_A} \quad (4.21)$$

where  $\bar{x}_A$  is the sample mean and  $s_A$  the sample standard deviation. It is the gold standard when data approximates a Gaussian distribution.

### 4.8.3 Decimal Scaling

Normalizes values by shifting the decimal point via powers of 10:

$$v' = \frac{v}{10^j} \quad (4.22)$$

where  $j$  is the smallest positive integer satisfying:

$$\max(|v'|) < 1 \iff 10^{j-1} \leq \max(|v|) < 10^j \quad (4.23)$$

#### Numerical Example: Decimal Scaling

Given the vector  $V = \{-10, 201, 301, -401, 501, 601, 701\}$ :

1. The absolute maximum is  $m = \max(|v|) = 701$ .
2. The minimal integer  $j$  where  $10^j > 701$  is  $j = 3$  ( $10^3 = 1000$ ).
3. Dividing each value by 1000 yields:

$$V' = \{-0.01, 0.201, 0.301, -0.401, 0.501, 0.601, 0.701\} \quad (4.24)$$

Every transformed element strictly lies within the open interval  $(-1, +1)$ .

## 4.9 Univariate Outlier Treatment

Operational treatment of isolated univariate outliers relies on formal detection and replacement criteria:

### 4.9.1 Interquartile Criterion (Tukey)

1. Compute interquartile range  $IQR = Q_3 - Q_1$ .



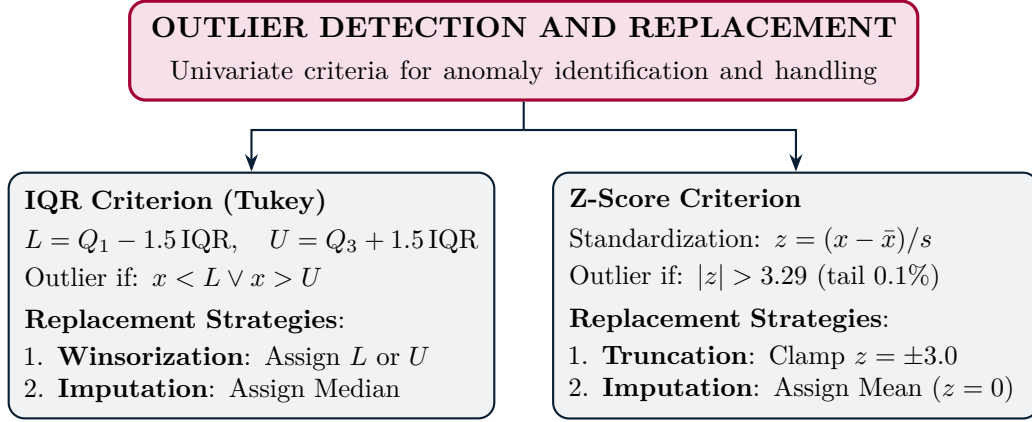


Figure 4.9: Decision hierarchy for univariate outlier detection and replacement strategies.

2. Define inner fences:

$$L = Q_1 - 1.5 \times \text{IQR}, \quad U = Q_3 + 1.5 \times \text{IQR} \quad (4.25)$$

3. Observation  $x$  is flagged as an outlier if  $x < L \vee x > U$ .

4. **Treatment:**

- *Winsorization:* Clamp  $x$  to  $L$  if  $x < L$ , or to  $U$  if  $x > U$ . Preserves ordinal rankings while neutralizing distortive tail extremes.
- *Median Imputation:* Replace flagged outliers with the sample median computed from non-outlier records.

#### 4.9.2 Z-Score Criterion

Under standard normal assumptions, 95% of samples fall within  $|z| \leq 1.96$ , 99% within  $|z| \leq 2.58$ , and 99.9% within  $|z| \leq 3.29$ . Flag as outliers instances exceeding:

$$|z| > 3.29 \iff x < \bar{x} - 3.29 s \quad \vee \quad x > \bar{x} + 3.29 s \quad (4.26)$$

#### Replacement Strategies

1. **3 $\sigma$  Truncation:** Clamp extreme values to boundary  $z = \pm 3.0$  ( $x' = \bar{x} \pm 3.0 s$ ), pulling observations into plausible bounds without erasing their status as extreme observations.

2. **Mean Imputation:** Substitute the outlier with  $z = 0$ , equivalent to the sample mean  $\bar{x}$ .

## Chapter 5

# Python Lab: Data Understanding and Data Preparation

This chapter presents the computational realization of the theoretical principles of Data Understanding (DU) and Data Preparation (DP), drawing upon the practical laboratory exercises implemented with Python's core data science ecosystem (`numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn`, and `scikit-learn`).

The laboratory uses the **Tips** dataset (augmented with the secondary **Tips\_Age** table), describing restaurant customer transactions and tipping behaviors, to demonstrate the complete data engineering workflow: from ingestion, relational joining, and deduplication to missing value diagnosis, conditional imputation, feature engineering, normalization, and dimensionality reduction via PCA and *t*-SNE.

### Interactive Environment and Laboratory Reproducibility

All examples, data preparation pipelines, and visualizations presented in this chapter are fully reproducible and interactively executable via Jupyter notebooks located in the `Data-Mining/notebooks/` directory.

#### Quick start with Docker:

- Terminal execution (project root): `docker compose up jupyter`
- Windows shortcut: double-click `scripts/start-jupyter.cmd`
- Web interface: <http://localhost:8888/lab> (no token or password required).

To execute headless Python scripts directly from the command line:

- `docker compose run -rm app python3 Data-Mining/notebooks/generate_dispensa_figures.py`

## 5.1 Environment, Ingestion, and Data Integration

**Remark 5.1.1** (Interactive Notebook Reference). Code examples in Sections 5.1, 5.2, and 5.3 correspond to the interactive Jupyter **Notebook 1 (Data Understanding)** (file `1_basics_and_understanding.ipynb`).

The scientific software stack employed throughout the laboratory exercises comprises:

```
1 import numpy as np
2 import pandas as pd
3 import scipy.stats as stats
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.impute import SimpleImputer, KNNImputer
7 from sklearn.preprocessing import StandardScaler,
   MinMaxScaler, KBinsDiscretizer, OneHotEncoder,
   TargetEncoder
8 from sklearn.decomposition import PCA
9 from sklearn.manifold import TSNE
```

### 5.1.1 Loading and Structural Audit

The primary dataset is ingested from a tab-separated values (`tsv`) file, setting the index to the first column:

```
1 tips = pd.read_csv('tips.csv', sep='\t', index_col=0)
2 print(tips.head())
3 tips.info()
```

The raw data initially contains 244 observations and 7 features: `total_bill` (total bill amount), `tip` (tip amount), `sex` (server/payer gender), `smoker` (smoking party indicator), `day` (day of week), `time` (meal timing: Lunch or Dinner), `size` (party size).

### 5.1.2 Data Integration and Relational Joining

In this lab, customer age data is stored in a separate table (`tips_age.csv`). The two sources are integrated using a left-join keyed on the shared primary identifier ID:

```
1 tips_age = pd.read_csv('tips_age.csv', sep='\t',
2                       index_col=0)
3 df = tips.join(tips_age, on='ID')
4 # Drop redundant columns originating from the secondary
   source
5 df = df.drop(columns=['gender'], errors='ignore')
```

### 5.1.3 Duplicate Row Detection and Filtering

Identifying duplicate rows requires inspecting exact equality across all attribute dimensions:

```
1 # Display all duplicate rows (keeping all duplicate
   instances)
2 duplicated_rows = df[df.duplicated(keep=False)]
3 print(f"Duplicate rows detected: {len(duplicated_rows)}")
4
5 # Drop duplicate rows while retaining the first
   occurrence
6 df = df.drop_duplicates()
```

### 5.1.4 Cleaning Invalid Values and Type Casting

Real-world datasets frequently contain formatting anomalies or sentinel strings mistakenly entered into numeric fields (such as the dummy string 'aaaaa' in the `total_bill` column). This forces Pandas to treat the column as a generic object type. Cleaning requires string replacement followed by explicit numerical casting:

```
1 # Replace invalid string with NaN
2 df["total_bill"] = df["total_bill"].replace("aaaaa", np.
      nan)
3
4 # Cast column to numeric float
5 df["total_bill"] = pd.to_numeric(df["total_bill"])
```

## 5.2 Missing Value Diagnosis and Imputation

### 5.2.1 Missing Value Audit

Missing values (NaN) are systematically audited by chaining `isnull()` and `sum()`:

```
1 # Summary of null counts per column
2 null_counts = df.isnull().sum()
3 print(null_counts[null_counts > 0])
4
5 # Filter records containing at least one missing
      attribute
6 incomplete_records = df[df.isnull().any(axis=1)]
```

### 5.2.2 Deletion vs. Statistical Imputation

- **Listwise Deletion:** Discarding all rows containing one or more null values:

```
1 df_cleaned = df.dropna()
```

- **Conditional Median Imputation:** When the `age` feature exhibits missing values or dummy zeros (0), global median imputation can mask real demographic disparities between meal times (*Lunch* vs. *Dinner*). Group-stratified imputation preserves local distributions:

```

1 df['age'] = df['age'].replace(0, np.nan)
2 # Impute using group-conditional median based on
  meal time
3 df['age'] = df['age'].fillna(df.groupby('time')['age']
  ).transform('median'))

```

### 5.2.3 Advanced Imputation with Scikit-Learn

The `sklearn.impute` module provides transformer objects suitable for production pipelines:

```

1 # 1. SimpleImputer for numerical attributes (mean or
  median)
2 imp_mean = SimpleImputer(strategy='mean')
3 df['tip'] = imp_mean.fit_transform(df[['tip']])
4
5 # 2. SimpleImputer for categorical attributes (most
  frequent / mode)
6 imp_mode = SimpleImputer(strategy='most_frequent')
7 df['smoker'] = imp_mode.fit_transform(df[['smoker']]).
  ravel()
8
9 # 3. KNNImputer (multivariate k-nearest neighbors
  imputation)
10 knn_imp = KNNImputer(n_neighbors=2, weights='distance')
11 num_cols = ['total_bill', 'tip', 'size', 'age']
12 df[num_cols] = knn_imp.fit_transform(df[num_cols])

```

## 5.3 Multivariate Exploration and Correlation Analysis

### 5.3.1 Contingency Tables and Conditional Distributions

Evaluating dependencies across categorical features relies on row-normalized cross-tabulations:

```

1 # Joint frequency table between smoker status and gender
2 smoker_xt = pd.crosstab(df['smoker'], df['sex'])
3

```

```

4 # Row-wise normalization (proportional fractions)
5 smoker_pct = smoker_xt.div(smoker_xt.sum(1).astype(float)
6                             ), axis=0)
7 smoker_pct.plot(kind='bar', stacked=False, title='Gender
8               per Smoker Category')
9 plt.ylabel('Relative Frequency')
10 plt.show()

```

### 5.3.2 Correlation Matrices and Heatmap Rendering

Pandas computes Pearson, Spearman, and Kendall correlation matrices across numeric attributes:

```

1 # Pearson linear correlation matrix
2 corr_pearson = df.corr(numeric_only=True, method='
3     pearson')
4
5 # Spearman rank correlation matrix (robust monotonic
6     measure)
7 corr_spearman = df.corr(numeric_only=True, method='
8     spearman')
9
10 # Annotated heatmap rendering via Seaborn
11 plt.figure(figsize=(8, 6))
12 sns.heatmap(corr_pearson, annot=True, cmap='coolwarm',
13             fmt='.2f', vmin=-1, vmax=1)
14 plt.title('Pearson Correlation Heatmap (Tips Dataset)')
15 plt.show()

```

### 5.3.3 Scatter Matrix and Pairplots

Pairwise bivariate inspections are efficiently visualized using `scatter_matrix` or Seaborn `pairplot`:

```

1 # Pandas Scatter Matrix with KDE diagonal
2 pd.plotting.scatter_matrix(df[['total_bill', 'tip', '
3     size', 'age']], figsize=(9, 9), diagonal='kde')
4 plt.show()
5
6 # Seaborn Pairplot with class-based color coding
7 sns.pairplot(df, vars=['total_bill', 'tip', 'size'], hue
8               ='time', palette='Dark2')
9 plt.show()

```



### 5.3.4 Feature Engineering: Domain Ratio Construction

Domain expertise guides the derivation of novel features with enhanced explanatory power:

```
1 # 1. Effective tip ratio relative to total bill
2 df['tip_ratio'] = df['tip'] / df['total_bill']
3
4 # 2. Bill per person (BPP)
5 df['bpp'] = df['total_bill'] / df['size']
6
7 # Stratified Boxplot inspection across gender
8 df.boxplot(column=['bpp'], by='sex')
9 plt.title('Bill per Person Stratified by Gender')
10 plt.suptitle('')
11 plt.show()
```

## 5.4 Feature Transformation, Discretization, and Encoding

**Remark 5.4.1** (Interactive Notebook Reference). Code examples in Sections 5.4 and 5.5 correspond to the interactive Jupyter [Notebook 2 \(Feature Engineering\)](#):

[2\\_feature\\_engineering\\_and\\_data\\_representation.ipynb](#)

### 5.4.1 Discretization with KBinsDiscretizer

Scikit-Learn's `KBinsDiscretizer` converts continuous attributes into discrete ordinal bins using three distinct binning strategies:

```
1 # Discretization into 4 bins across different strategies
2 disc_uniform = KBinsDiscretizer(n_bins=4, encode='
    ordinal', strategy='uniform')      # Equal-Width
3 disc_quantile = KBinsDiscretizer(n_bins=4, encode='
    ordinal', strategy='quantile')     # Equal-Frequency
4 disc_kmeans = KBinsDiscretizer(n_bins=4, encode='ordinal
    ', strategy='kmeans')              # Cluster-based
5
6 df['total_bill_bins'] = disc_uniform.fit_transform(df[['
    total_bill']])
7 print(df['total_bill_bins'].value_counts())
```

### 5.4.2 Feature Scaling: StandardScaler vs. MinMaxScaler

- **StandardScaler**: Standardizes features to zero mean and unit variance ( $\mu = 0, \sigma = 1$ ).
- **MinMaxScaler**: Bounds values strictly into  $[0, 1]$ .

```
1 num_cols = ['total_bill', 'tip', 'age']
2
3 # Z-Score Standardization
4 scaler_std = StandardScaler()
5 df_standardized = df.copy()
6 df_standardized[num_cols] = scaler_std.fit_transform(df[
    num_cols])
7
8 # Min-Max Normalization
9 scaler_minmax = MinMaxScaler()
10 df_normalized = df.copy()
11 df_normalized[num_cols] = scaler_minmax.fit_transform(df[
    num_cols])
12
13 # Inverting transformation back to original scale
14 df_recovered = scaler_minmax.inverse_transform(
    df_normalized[num_cols])
```

### 5.4.3 Categorical Encoding: One-Hot vs. Target Encoding

```
1 # 1. One-Hot Encoding (prevents imposing synthetic order
2 )
3 ohe = OneHotEncoder(handle_unknown='ignore',
4     sparse_output=False)
5 encoded_days = ohe.fit_transform(df[['day']])
6 encoded_df = pd.DataFrame(encoded_days, columns=ohe.
7     get_feature_names_out(['day']))
8
9 # 2. Target Encoding (substitutes categories with target
10 expectation)
11 target_enc = TargetEncoder(smooth='auto')
12 # Encode 'day' using mean 'tip'
```

```

9 df['day_target_encoded'] = target_enc.fit_transform(df[['
    'day']], df['tip'])

```

## 5.5 Dimensionality Reduction: PCA and *t*-SNE

### 5.5.1 Principal Component Analysis (PCA)

Features must be centered and scaled before fitting PCA so that high-variance variables do not arbitrarily dominate the spectral decomposition:

```

1 # Select quantitative predictors
2 X = df[['total_bill', 'tip', 'size', 'age', 'tip_ratio',
    'bpp']].dropna()
3 X_scaled = StandardScaler().fit_transform(X)
4
5 # Fit PCA
6 pca = PCA()
7 X_pca = pca.fit_transform(X_scaled)
8
9 # Eigenvalues (absolute variance explained per component
    )
10 eigenvalues = pca.explained_variance_
11 # Relative variance ratio
12 explained_var_ratio = pca.explained_variance_ratio_
13 # Cumulative explained variance
14 cumulative_var = np.cumsum(explained_var_ratio)

```

### Scree Plot Visualization

The Scree Plot helps detect the characteristic "elbow" where incremental principal components contribute negligible variance:

```

1 plt.figure(figsize=(8, 4))
2 plt.plot(range(1, len(eigenvalues) + 1), eigenvalues, 'o
    -', color='navy', label='Individual Eigenvalues')
3 plt.title('PCA Scree Plot (Eigenvalues)')
4 plt.xlabel('Principal Component Index')
5 plt.ylabel('Eigenvalue (Explained Variance)')
6 plt.grid(True, linestyle='--', alpha=0.5)
7 plt.legend()
8 plt.show()

```

### 5.5.2 Non-linear Projection via *t*-SNE

*t*-Distributed Stochastic Neighbor Embedding (*t*-SNE) is an unsupervised non-linear algorithm designed to preserve local neighborhood affinities in high-dimensional datasets:

```
1 tsne = TSNE(  
2     n_components=2,  
3     perplexity=30,          # Balance between local and  
4     global structure (typical range: 5-50)  
5     learning_rate=200,      # Gradient descent step size  
6     max_iter=1000,          # Maximum KL-divergence  
7     optimization iterations  
8     random_state=42  
9 )  
10 X_tsne = tsne.fit_transform(X_scaled)
```

### 5.5.3 Methodological Comparison: PCA vs. *t*-SNE

```
1 # Side-by-side comparative scatter plots  
2 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))  
3  
4 # PCA projection  
5 ax1.scatter(X_pca[:, 0], X_pca[:, 1], c='navy', alpha  
6             =0.7, edgecolors='k')  
7 ax1.set_title('Linear PCA Projection (PC1 vs PC2)')  
8 ax1.set_xlabel('First Principal Component')  
9 ax1.set_ylabel('Second Principal Component')  
10 ax1.grid(True, linestyle='--', alpha=0.5)  
11  
12 # t-SNE projection  
13 ax2.scatter(X_tsne[:, 0], X_tsne[:, 1], c='crimson',  
14            alpha=0.7, edgecolors='k')  
15 ax2.set_title('Non-linear t-SNE Projection')  
16 ax2.set_xlabel('t-SNE Dimension 1')  
17 ax2.set_ylabel('t-SNE Dimension 2')  
18 ax2.grid(True, linestyle='--', alpha=0.5)  
19  
20 plt.tight_layout()  
21 plt.show()
```

| Criterion                   | PCA (Principal Component Analysis)                                            | $t$ -SNE ( $t$ -Distributed SNE)                                                              |
|-----------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Nature</b>               | Linear, global, deterministic.                                                | Non-linear, local, stochastic/probabilistic.                                                  |
| <b>Objective</b>            | Maximizes projected global variance via eigenvalue decomposition.             | Minimizes Kullback-Leibler divergence between neighborhood probability distributions.         |
| <b>Interpretability</b>     | Component axes are linear feature combinations with explicit loadings.        | Axes possess no absolute physical meaning; only relative neighbor proximity matters.          |
| <b>Long-Range Distances</b> | Preserves large-scale Euclidean spatial configurations.                       | Distorts global inter-cluster distances (cluster radii do not represent true cluster volume). |
| <b>Standard Use</b>         | Dimensionality reduction for supervised learning, compression, decorrelation. | Exploratory manifold visualization and qualitative cluster identification (2D/3D).            |

Table 5.1: Comparative synthesis of PCA and  $t$ -SNE characteristics.

# Bibliography